University of Sousse

National Engineering School of Sousse

Department of Advanced Mechanics

End of Study Project Report

Submitted in partial fulfilment of the requirements for the degree of

Engineer in Mechatronics Engineering

Design and Build of a Self-Contained Winch for a Spatial Cable-Driven Parallel Robot

Prepared by:

Ms. Salsabil HACHEM

Defended on **16/09/2026** before the jury:

Chair	:	Dr. Farhat Zemzemi, ENISo
Reviewer	:	Dr. Radhouane Sghir, ENISo
Academic supervisor	:	Dr. Houssem Saafi, ENISo
Professional supervisor	:	Dr. Mitchell Rushton, Ontario Tech

Academic year 2025–2026

Approval

This report has been examined and approved by:

Professional Supervisor

Dr. Mitchell Rushton

Mitchell Rushton
Digitally signed by Mitchell Rushton
Date: 2026.09.04 09:07:03 -04'00'

Signature and date

Academic Supervisor

Dr. Houssem Saafi



Signature and date

Dedication

﴿وَآخِرُ دَعْوَاهُمْ أَنِ الْحَمْدُ لِلَّهِ رَبِّ الْعَالَمِينَ﴾

To my parents, **Hedi** *and* **Monia**,
for the years of patience, for the trust you never withdrew,
and for the countless sacrifices you chose to make quietly.
Nothing in these pages would exist without you.

To my sisters, **Sinda** *and* **Serine**,
for the laughter that lightened the hardest weeks,
for listening without judging, and for reminding me
that I was never facing this alone.

To my **family**,
near and far, for the encouragement, the prayers,
and the constant presence at every stage of this journey.

To my **friends**,
for the shared effort, the honest advice,
and the support that carried me through this final year.

To every **teacher** *who gave me*
not only knowledge, but the discipline to use it well.

And to everyone who, in a word or a gesture,
helped me move one step further.

Salsabil

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my professional supervisor, **Dr. Mitchell Rushton**, and my academic supervisor, **Dr. Houssem Saafi**, for their continuous guidance, patience, and valuable advice throughout this project.

I am deeply grateful to **Ontario Tech University** and the **Mitacs Globalink Research** program for giving me the opportunity to conduct this research in a stimulating and international research environment.

I would like to warmly thank all the members of the **Mechatronics Laboratory** at Ontario Tech University for their help and their availability throughout this internship.

I would also like to thank the members of the jury for their time, for accepting to evaluate my work, and for their insightful remarks during my defense.

Special thanks to all the professors and staff at the **National Engineering School of Sousse (ENISO)** for the quality of their teaching and the technical skills they have shared with me over the years.

Salsabil HACHEM

Abstract

Cable-driven parallel robots replace rigid links by flexible cables, which requires every cable to remain under a controlled positive tension. In conventional winches, the winding radius varies as the cable stacks unevenly, and the tension is most often neither measured at the actuator nor regulated.

This work presents the design and development of a self-contained winch unit for a spatial cable-driven parallel robot, carried out at the Mechatronics Laboratory of Ontario Tech University under the Mitacs Globalink Research Internship program. A rototranslating drum with a helical groove guarantees a constant winding radius over the 5 m of cable, a direct-drive brushless motor driven over CAN-FD provides the actuation without backlash, and a measuring pulley carried by a load cell inside the unit gives the cable tension without any connection to the moving platform. Only two electrical wires leave the unit, the power supply and the bus, so that adding a winch to the robot means adding a node on the existing network.

A tension regulation and a hybrid control strategy blending a length command and a tension command were implemented in real time on the prototype and tested on a bench where a suspended mass imposes a known tension. The result is a modular actuator in which the cable tension is measured and regulated locally, inside the unit itself.

Keywords: Cable-driven parallel robot (CDPR), self-contained winch, grooved drum, tension control, direct-drive motor, real-time control, CAN-FD.

List of Figures

1.1 Mitacs Globalink Research Internship logo [11] . . . 3
1.2 Ontario Tech University logo [12] . . . 4
1.3 The Mechatronics Laboratory of Ontario Tech University . . . 4
1.4 Needs analysis diagram of the self-contained winch . . . 6
1.5 Gantt chart of the project . . . 7

2.1 Typical manipulator architectures . . . 10
2.2 Concept and components of a cable-driven parallel robot . . . 10
2.3 Vector loop of the inverse kinematics . . . 12
2.4 Cable configurations of a CDPR . . . 15
2.5 Examples of CDPR applications . . . 16
2.6 Rotating drum configuration . . . 17
2.7 Rototranslating drum actuation unit . . . 17
2.8 Drum with spooling helper . . . 18

3.1 Existing planar CDPR setup . . . 21
3.2 Winch unit of the existing setup . . . 22
3.3 mj5208 brushless motor [1] . . . 23
3.4 moteus r4.11 motor controller [2] . . . 25
3.5 mjcanfd-usb-1x CAN-FD to USB adapter [3] . . . 25
3.6 Wheatstone bridge of a strain gauge load cell . . . 26
3.7 CZL601 miniature load cell [4] . . . 28
3.8 HX711 module . . . 28
3.9 Arduino Mega board . . . 29
3.10 Candidate pulley geometries for the integration of the load cell . . . 30

3.11 Overall geometric layout of the winch unit 31

4.1 Full design of the self-contained winch unit 33
4.2 Section view of the winch unit 34
4.3 Components of the winch unit 34
4.4 Helical groove profile 35
4.5 Drum and groove parameters 36
4.6 CAD model of the grooved drum 37
4.7 Section view of the grooved drum 38
4.8 The measuring pulley design 38
4.9 Swivel pulley assembly 39
4.10 Load cell support holding the sensor at 45° 40
4.11 Base plate of the winch unit 41
4.12 Tr8×2 trapezoidal lead screw [5] 41
4.13 SK8 shaft support block 42
4.14 Linear guide rod ∅6 × 200 mm [6] 42
4.15 Self-lubricating sleeve bushing 43
4.16 Flexible shaft coupling 6–8 mm 43
4.17 Target robot: eight winch units on the pegboard frame 44

5.1 Prusa CORE One 3D printer 46
5.2 Assembled prototype of the self-contained winch unit 46
5.3 Control structure of the moteus firmware 47
5.4 Connectors of the moteus r4.11 controller 49
5.5 Wiring of the motor and interface with the host computer 50
5.6 Interface between the motor and the host computer, reached through the mjcanfd-usb-1x adapter 50
5.7 Graphical user interface `tview` 51
5.8 Position response recorded in tview 53
5.9 Position control of the motor from the host script 54
5.10 Wiring of the tension acquisition chain 55
5.11 Calibrated readings recorded with the two reference weights 57
5.12 General view of the experimental setup 57

6.1 Test bench: tension applied by a suspended mass 60
6.2 Block diagram of the tension control loop 61
6.3 Block architecture of the control software 62
6.4 Steps executed at each control cycle 64
6.5 Tension regulated at the set-point $T^* = 1.960$ N for a suspended mass of 200 g 66
6.6 Block diagram of the hybrid control loop 67
6.7 Evolution of the position and tension weights with the position error . . . 67
6.8 Steady state of the hybrid controller 70
6.9 Position and tension during a hybrid control test 70
B.1 Dimensions of the Tr8×2 lead screw and its brass nut [5] 86
B.2 Mechanical drawing of the mj5208 motor [1] 87
B.3 Dimensions and characteristics of the load cell [4] 87
B.4 Wiring of the load cell [4] 88

List of Tables

1.1 Specifications of the winch unit 6

2.1 Notation used for the modelling of a CDPR 12
2.2 Concepts for integrating a one-axis force sensor into a cable robot. Adapted from [7]. 19

3.1 Comparison of the candidate winch architectures 23
3.2 mj5208 motor specifications [1] 23
3.3 moteus r4.11 controller specifications [2] 25
3.4 Strain gauge load cell geometries. Adapted from [8]. 27
3.5 CZL601 load cell specifications [4] 28
3.6 HX711 specifications [9] 29

4.1 Drum and groove parameters 36
4.2 Summary of the drum dimensioning 37
4.3 Ball bearings used in the winch unit 43

5.1 Control modes of the moteus controller [10] 48
5.2 Configuration parameters of the moteus controller 52
5.3 Limits sent with each position command 54
5.4 Load cell, HX711 and Arduino wiring connections 55
5.5 Identified constants and settings of the measurement chain 56
5.6 Comparison between the reference weights and the measured values 57

6.1 Description of the three concurrent software blocks 63
6.2 Parameters of the tension controller 65
6.3 Safety mechanisms of the tension control system 65

6.4 Parameters of the hybrid controller 69
6.5 Additional safety mechanisms of the hybrid controller 70

List of Abbreviations

ADC	Analog-to-Digital Converter
BLDC	Brushless Direct Current (motor)
CAD	Computer-Aided Design
CAN	Controller Area Network
CAN-FD	Controller Area Network with Flexible Data-rate
CDPR	Cable-Driven Parallel Robot
DOF	Degree of Freedom
ENISo	National Engineering School of Sousse
FDM	Fused Deposition Modelling
FEAS	Faculty of Engineering and Applied Science
FOC	Field-Oriented Control
F.S.	Full Scale (load cell specification)
GRI	Globalink Research Internship
ISO	International Organization for Standardization
LSB	Least Significant Bit
PI	Proportional–Integral (controller)
PID	Proportional–Integral–Derivative (controller)
PLA	Polylactic Acid
PWM	Pulse-Width Modulation
SPS	Samples Per Second
SVPWM	Space-Vector Pulse-Width Modulation
UI	User Interface
USB	Universal Serial Bus

Contents

Dedication ii

Acknowledgements iii

Abstract iv

List of Figures v

List of Tables viii

List of Abbreviations x

General Introduction 1

1 General Presentation and Project Context 3

Introduction . . . 3

1.1 Host Institutions . . . 3

1.1.1 Mitacs Globalink Research . . . 3

1.1.2 Ontario Tech University . . . 3

1.1.3 The Mechatronics Laboratory . . . 4

1.2 Project Framework . . . 4

1.2.1 Problem Statement . . . 4

1.2.2 Project Objectives . . . 5

1.2.3 Specifications . . . 6

1.2.4 Needs Analysis . . . 6

1.2.5 Work Planning . . . 7

Conclusion . . . 8

2 Literature Review **9**

2.1 Robotic Manipulator Architectures 9

2.2 Cable-Driven Parallel Robots 10

2.2.1 General Overview 10

2.2.2 Advantages and Limitations 10

2.2.3 Kinematic and Static Modelling 12

2.2.4 Classification by Workspace 14

2.2.5 Classification by Kinematic Configuration 14

2.2.6 Applications of CDPRs 15

2.3 Cable Actuation Units 16

2.3.1 Rotating Drum 16

2.3.2 Rototranslating Drum 17

2.3.3 Drum with Spooling Helper 17

2.3.4 Towards Self-Contained Winch Units 18

2.4 Control and Tension Sensing in CDPRs 18

2.4.1 Control Strategies 18

2.4.2 Tension Sensing 19

Conclusion 20

3 Design Study and Component Selection **21**

Introduction 21

3.1 Analysis of the Existing Winch 21

3.2 Selection of the Winch Architecture 22

3.3 Component Selection 23

3.3.1 Actuator Selection 23

3.3.2 Motor Controller 24

3.3.3 Communication Interface 25

3.3.4 Tension Sensor Selection 25

3.3.5 Signal Conditioning 28

3.3.6 Microcontroller Selection 29

3.4 Geometric Design 29

3.4.1 Pulley Geometry Selection 29

3.4.2 Overall Winch Layout 31
Conclusion 31

4 Mechanical Design **33**
Introduction 33
4.1 Overview of the Winch Unit 33
4.2 Drum Design 35
4.2.1 Helical Groove Geometry 35
4.2.2 Drum Dimensioning 35
4.3 Pulley Design 38
4.3.1 Measuring Pulley 38
4.3.2 Swivel Pulley 39
4.4 Load Cell Support 40
4.5 Base Plate 40
4.6 Mechanical Transmission Components 41
4.7 Integration on the Target Robot 44
Conclusion 44

5 Prototyping, Electrical Integration and Calibration **45**
Introduction 45
5.1 Prototyping and Assembly 45
5.1.1 3D Printing 45
5.1.2 Assembled Prototype 46
5.2 Motor Actuation Chain 46
5.2.1 Control Structure of the Controller 47
5.2.2 Wiring and Interface with the Host Computer 48
5.2.3 Diagnostic and Configuration 50
5.2.4 Motor Control 52
5.3 Tension Acquisition Chain 54
5.3.1 Load Cell and HX711 Wiring 54
5.3.2 Calibration Principle 55
5.3.3 Calibration Results 56

5.4 Overview of the Experimental Setup 57
Conclusion 58

6 Control System **59**
6.1 Tension Control 59
6.1.1 Control Strategy 59
6.1.2 PI Controller 61
6.1.3 Software Implementation 62
6.1.4 Safety Management 65
6.1.5 Experimental Results 66
6.2 Hybrid Control 66
6.2.1 Control Strategy 66
6.2.2 Weighting Function 67
6.2.3 Position and Tension Controllers 68
6.2.4 Implementation 69
6.2.5 Safety Management 69
6.2.6 Experimental Results 70

General Conclusion **72**

Bibliography **74**

Appendices **77**

Appendix A — Source Code **78**

Appendix B — Component Datasheets **86**

General Introduction

A **cable-driven parallel robot** (CDPR) is a parallel manipulator in which the rigid links are replaced by flexible cables. A moving platform is connected to a fixed frame by several cables, each wound on its own motorised winch, and the pose of the platform is obtained by controlling the length of each cable. Since a cable is light and can be wound over great lengths, the workspace of such a robot is limited by the size of its frame rather than by the mass of its structure: workspaces of several hundred metres are reached, and payloads of several tonnes are carried, at a scale that no serial or rigid-link parallel architecture reaches. This property explains the applications of CDPRs at large scale — camera systems above stadiums, handling in warehouses, large-scale additive manufacturing, rehabilitation devices and the inspection of large structures.

This advantage comes with a constraint that governs the whole design: a cable can only **pull**, never push. Every cable must therefore remain under tension at all times, and that tension must stay below the strength of the cable. Because of this unilateral behaviour, a cable robot generally needs more cables than the number of degrees of freedom it controls. It is then **redundantly actuated**, and the way the load is shared between the cables becomes a variable of the control: it is this redundancy that allows every cable to be kept taut over the whole workspace.

Two quantities therefore have to be mastered on each cable, and both are produced by the same component. The first is the **length**, from which the pose of the platform is deduced by the kinematic model. The second is the **tension**, which must be kept inside its bounds while the platform moves. The component that delivers the length and carries the tension is the **actuation unit** — the winch. It therefore fixes, by itself, three characteristics of the complete robot: the accuracy with which a cable length is known, the tension range that can be applied, and the maximum speed of the platform.

The approach followed in this work is a **self-contained winch**: a module that gathers inside a single envelope the motor, its controller, the storage and the guidance of the cable, and the measurement of its tension. Such a unit needs only a power supply and a communication bus, and the robot is then built by replicating it.

This project was carried out at the **Mechatronics Laboratory of Ontario Tech University**, in the framework of a **Mitacs Globalink Research Internship**. The

objective is to design, build and test one such winch, intended to equip a cable-driven parallel robot built on a pegboard frame, and to validate on it the control strategies that the robot will require.

The report follows this objective in six chapters:

- **Chapter 1** presents the host laboratory, the framework of the internship and the objectives of the project.
- **Chapter 2** reviews cable-driven parallel robots and their actuation units, and derives the requirements the winch must satisfy.
- **Chapter 3** establishes the specifications and selects the components: the actuator, its controller, the cable and the tension sensor.
- **Chapter 4** details the mechanical design of the unit and the integration of eight units on the target robot.
- **Chapter 5** describes the building and the assembly of the prototype, then the wiring, the diagnostic and the calibration of its two electrical chains.
- **Chapter 6** presents the control strategies implemented on the unit — a tension regulation and a hybrid strategy blending length and tension — and their experimental evaluation.

A general conclusion closes the report by summarising the results obtained .

Chapter 1

General Presentation and Project Context

Introduction

This chapter situates the project within its institutional and technical framework. It begins with a presentation of the host organizations: the **Mitacs Globalink** program, **Ontario Tech University**, and the **Mechatronics Laboratory**. The project is then defined through its problem statement, its objectives and its specifications, highlighting the mechatronic challenges addressed during this internship.

1.1 Host Institutions

1.1.1 Mitacs Globalink Research

Mitacs is a Canadian national research organization that fosters partnerships between academia, industry, and government. Founded in 1999, it supports innovation by funding research internships for students worldwide [11].

The **Mitacs Globalink Research Internship (GRI)** is a competitive initiative that matches top-ranked international students with Canadian research professors. This program provided the framework for the present research project.

Figure 1.1: Mitacs Globalink Research Internship logo [11]

1.1.2 Ontario Tech University

Located in Oshawa, Ontario, **Ontario Tech University** is a leading Canadian institution known for its focus on technology and innovation [12]. The internship was conducted within the **Faculty of Engineering and Applied Science (FEAS)**, which provides a

high-level research environment equipped with modern laboratories.

Figure 1.2: Ontario Tech University logo [12]

1.1.3 The Mechatronics Laboratory

The research work was carried out in the **Mechatronics Laboratory**, which is dedicated to the study of advanced mechanical systems and innovative robotic architectures. Its research focuses on:

- kinematics and dynamics of robotic systems;
- **cable-driven parallel robots (CDPRs)**;
- control systems for precise positioning.

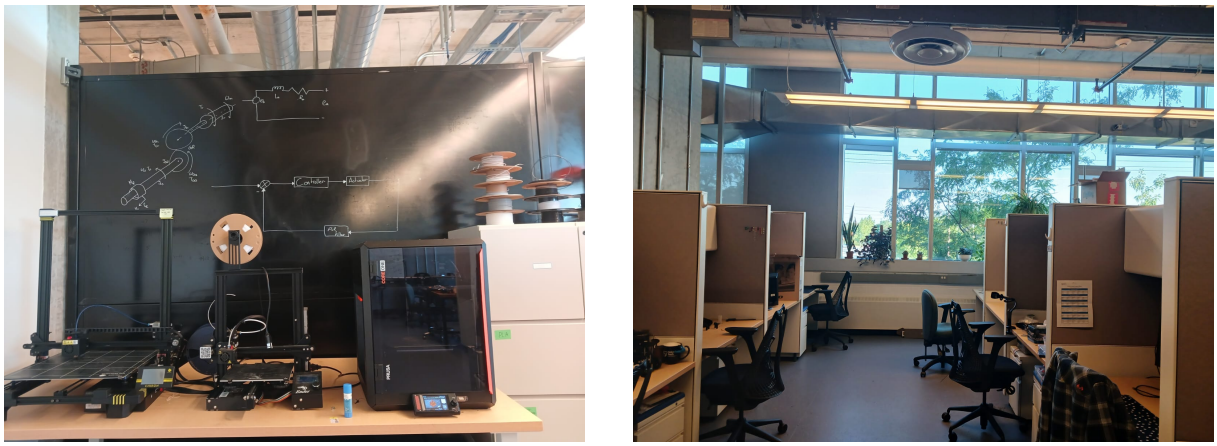


Figure 1.3: The Mechatronics Laboratory of Ontario Tech University

1.2 Project Framework

1.2.1 Problem Statement

The cable robot currently available in the host laboratory is **planar**; the objective is a **spatial robot** that controls the six degrees of freedom of the platform with eight cables on a pegboard table, each wound by its own winch. The move to a spatial robot changes

one thing at the winch: the cable no longer pulls in a fixed plane, so the winch must follow a direction that varies in three dimensions without displacing the point where the cable leaves it, which is the role of the swivel pulley designed in Chapter 4. Three further limitations of the existing winches, which constrain planar and spatial robots alike, stand in the way:

- **Variable winding radius:** the cable stacks in several layers.
- **Moving cable exit point:** the cable advances along a fixed drum, so the point where it leaves the winch moves, while the kinematic model assumes it fixed.
- **No tension sensor:** the winch carries no force sensor, so the tension is controlled only indirectly.

The problem addressed in this project is therefore to design a winch that removes these three limitations at the source, inside a single module.

1.2.2 Project Objectives

To address the challenges identified above, the project objectives are organized into two axes to ensure a comprehensive mechatronic development.

1. Mechanical and hardware design

- Model the winch structure in **SolidWorks**.
- Design a **helical-groove spool system** for a 5 m cable of 1 mm diameter, ensuring a constant, known winding radius and eliminating uneven cable layering.
- Integrate a **direct-drive BLDC motor** for high torque density, responsiveness, and mechanical compactness.

2. Control and software

- Implement a **real-time regulation loop** coordinating cable length and cable tension control.
- Develop an active **tension control strategy**.
- Integrate real-time **load cell (tension) and encoder (length) sensing** into the control loop for accurate closed-loop regulation.

1.2.3 Specifications

The requirements to be met by the new winch are given by the project and are summarised in Table 1.1. They constitute the reference against which the winch is dimensioned in Chapter 3 and evaluated in Chapter 6.

Table 1.1: Specifications of the winch unit

Requirement	**Criterion**	**Target value**
Cable storage	Length stored in a single layer	5 m
Cable	Diameter	1 mm
Cable exit	Position of the exit point	fixed
Tension range	Operating range	1–20 N
Interfaces	Cables leaving the unit	CAN-FD
Mounting	Interface with the frame	pegboard, 1/4″

1.2.4 Needs Analysis

The needs analysis diagram (Figure 1.4) identifies the objective of the system to be developed by answering the three fundamental questions of the method.

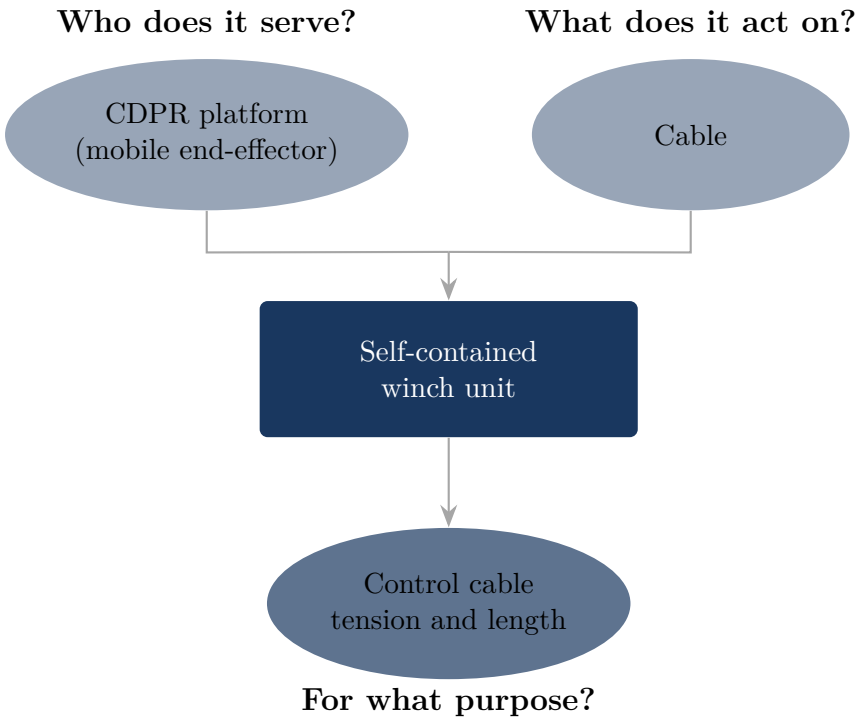


Figure 1.4: Needs analysis diagram of the self-contained winch

1.2.5 Work Planning

The internship lasted six months. Figure 1.5 covers the whole project rather than the internship alone. The tasks are the following:

- **State of the art:** literature review of cable-driven parallel robot architectures, winch designs, and actuation technologies.
- **Component selection:** evaluation and selection of the motor, the motor controller, and the load cell.
- **Mechanical design:** design of the drum, the housing, and the mounting structure using SolidWorks.
- **Prototyping and fabrication:** 3D printing of the drum prototype and assembly of the mechanical structure.
- **Sensor integration and calibration:** integration and calibration of the load cell for accurate real-time tension measurement.
- **Control architecture design:** development of the real-time control loop, including motor control (moteus / CAN-FD).
- **Implementation and testing:** implementation of the control code and experimental validation.
- **Report writing:** documentation and writing of the report.

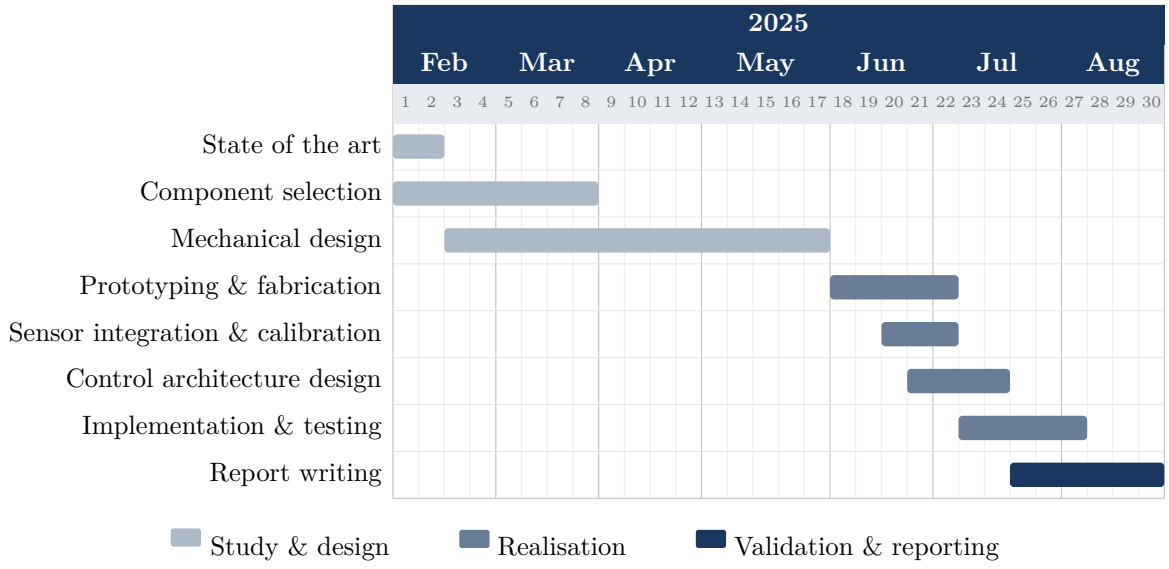


Figure 1.5: Gantt chart of the project, in weeks

Conclusion

This chapter has established the institutional and technical framework of the project, presenting the host institutions, the problem statement, the objectives and the specifications that guide this work. By highlighting the limitations of conventional winch designs — the variable winding radius, the moving cable exit point and the absence of a direct tension measurement — the technical requirements for a self-contained winch unit suited to spatial CDPR applications have been defined. The next chapter presents the **state of the art**, reviewing existing robotic architectures and cable-driven parallel robot technologies in order to establish the theoretical foundations of this project.

Chapter 2

Literature Review

This chapter presents the theoretical foundations and state of the art related to the design of a self-contained winch unit for a spatial cable-driven parallel robot. It begins with an overview of robotic manipulator architectures, then focuses on cable-driven parallel robots, their advantages and limitations, their kinematic and static modelling, and their classification by workspace and cable configuration. The state of the art of cable actuation units and of the control and tension sensing strategies used in CDPRs is finally reviewed, in order to situate the winch developed in this work.

2.1 Robotic Manipulator Architectures

Robotic manipulators are mechanical systems designed to perform tasks by moving an end-effector through space. Three typical configurations are considered: serial, parallel, and cable-driven parallel robots. These architectures differ fundamentally in their kinematic structure, actuation strategy, and performance characteristics.

Serial manipulators (Figure 2.1a) consist of an open kinematic chain, where each joint supports all subsequent links. This configuration offers a large workspace but suffers from accumulated inertia and positioning errors, as each joint's error propagates to the end-effector.

Parallel manipulators (Figure 2.1b), in contrast, use a closed kinematic chain with multiple independent chains connecting the base to the end-effector simultaneously. Since actuators are located near the base, parallel architectures offer significantly higher stiffness and precision, at the cost of a more limited workspace.

Cable-driven parallel robots (Figure 2.1c) extend the parallel manipulator concept by replacing rigid links with flexible cables, connecting the frame to the end-effector through multiple cable-actuated anchor points. This substitution significantly reduces the moving mass and extends the achievable workspace, while preserving the high stiffness and precision benefits of parallel actuation, forming the foundation of the system studied throughout this report.

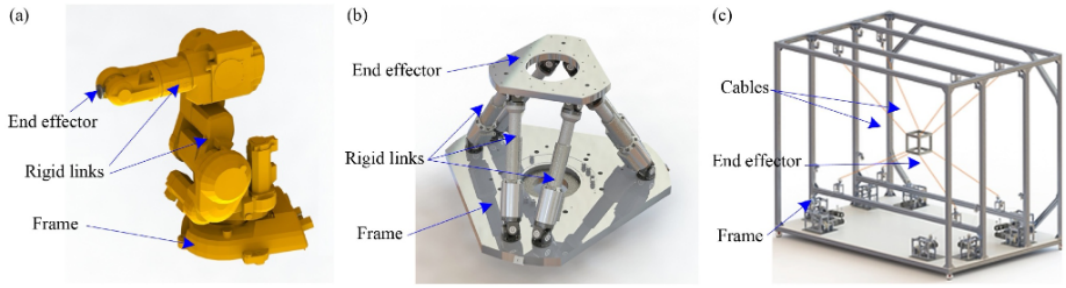


Figure 2.1: Typical configurations of (a) serial robot, (b) parallel robot, and (c) cable-driven parallel robot [13]

2.2 Cable-Driven Parallel Robots

2.2.1 General Overview

Cable-driven parallel robots represent a specific class of parallel manipulators where rigid links are replaced by flexible cables. These cables are wound around motorized spools and connected to a mobile platform or end-effector. By coordinating the length and tension of each cable, the position and orientation of the platform can be controlled with high precision [7].

The fundamental principle of CDPRs is based on parallel kinematics. The actuators are distributed at fixed anchor points, and the cables transmit forces to the end-effector. This configuration combines the advantages of parallel architectures with the specific benefits of cable actuation.

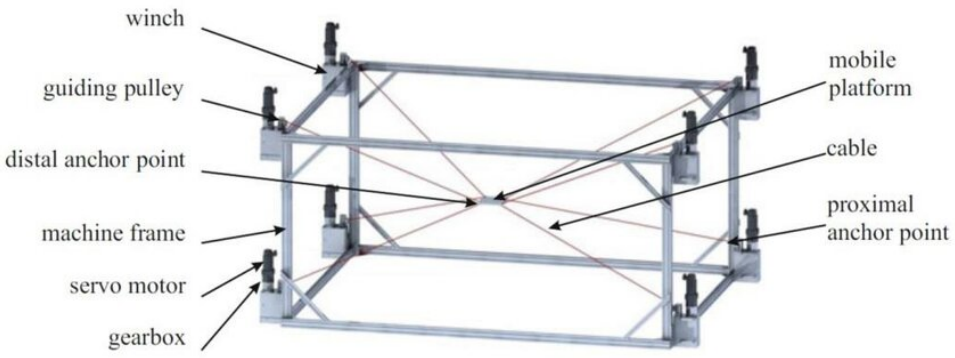


Figure 2.2: Concept and components of a cable-driven parallel robot, reproduced from [7]

2.2.2 Advantages and Limitations

The replacement of rigid links by cables gives CDPRs a set of properties that no other architecture combines [13, 14]:

- **Very large workspace.** The workspace is limited only by the length of the cables

and by the size of the frame, which allows structures of several tens of metres to be built at a reasonable cost.

- **Low moving mass and high dynamic performance.** Only the platform and the cables move, so the inertia is very low compared with a serial or a rigid parallel manipulator. This allows high accelerations, reduces the required actuator power, and makes CDPRs particularly suited to high-speed tasks.
- **Low cost and simple mechanical structure.** The cables replace machined links and articulated joints, so the frame reduces to a set of anchor points and the manufacturing cost grows far more slowly with the size of the robot than for a rigid architecture.
- **Modularity and easy reconfiguration.** The winches are independent modules attached to the frame, so the geometry of the robot, and therefore its workspace, can be modified simply by relocating them, without redesigning the mechanical structure.
- **High payload-to-mass ratio.** The load is distributed over several cables, each of which can carry a force far larger than its own weight.

These properties are obtained at the price of four limitations, which are the ones that constrain the design of the actuation unit [7, 15]:

- **Unilateral actuation.** A cable can only pull, never push. Every cable must therefore remain taut at all times, which forbids a purely geometric control of the robot and makes the tension a variable that has to be managed permanently.
- **Actuation redundancy.** Because of this unilateral behaviour, more cables than degrees of freedom are needed. The distribution of the load between the cables is then not unique and must be recomputed in real time.
- **Elasticity and sagging of the cables.** The cable elongates under tension and sags under its own weight. The length delivered by the winch is therefore not exactly the distance between the exit point and the platform, which limits the accuracy obtained by length control alone.
- **Cable interference.** The cables occupy the workspace: they can collide with each other, with the platform or with the environment, which restricts the usable part of the workspace, in particular for large orientations of the platform.

2.2.3 Kinematic and Static Modelling

The properties described above can be expressed formally, which makes it possible to quantify them and to justify the classification introduced in the next section. The notation used throughout this report is summarised in Table 2.1.

Table 2.1: Notation used for the modelling of a CDPR

Symbol	Definition
m	Number of cables
n	Number of degrees of freedom of the moving platform
$\mathbf{a}_i$	Position of the cable exit point i, in the fixed frame
$\mathbf{b}_i$	Position of the cable attachment point i, in the platform frame
$\mathbf{p}$	Position of the platform centre, in the fixed frame
$\mathbf{R}$	Rotation matrix of the platform
l_i	Length of cable i
$\mathbf{u}_i$	Unit vector along cable i, directed towards the exit point
τ_i	Tension in cable i
$\boldsymbol{\tau}$	Vector gathering the m cable tensions
τ_{min}, τ_{max}	Lower and upper admissible bounds of the cable tension
$\mathbf{W}$	Structure matrix of the robot
$\mathbf{w}_e$	External wrench applied to the platform (weight, payload)

Figure 2.3 shows the geometry of a CDPR.

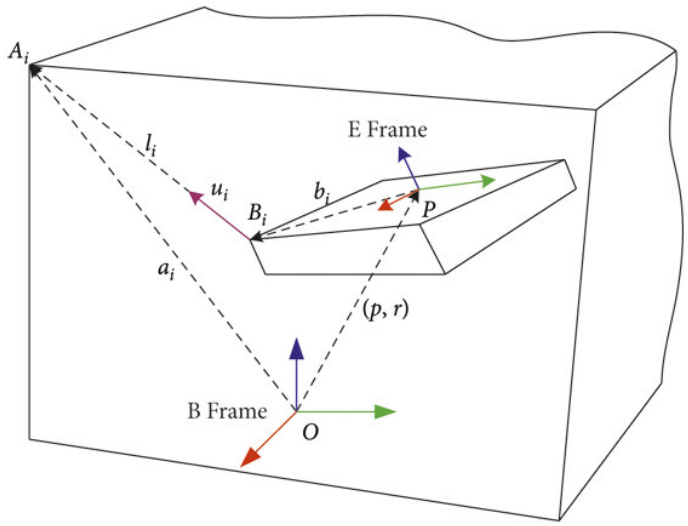


Figure 2.3: Vector loop of the inverse kinematics, reproduced from [15].

- **Inverse kinematics.** For a given pose $(\mathbf{p}, \mathbf{r})$ of the platform, the vector running from the attachment point i to the corresponding exit point is $\mathbf{a}_i - \mathbf{p} - \mathbf{R}\,\mathbf{b}_i$ (Figure 2.3). The length of cable i is its norm:

$$l_i = \|\mathbf{a}_i - \mathbf{p} - \mathbf{R}\,\mathbf{b}_i\|\,, \qquad i = 1, \ldots, m \tag{2.1}$$

and the unit vector giving the direction of the cable is

$$\mathbf{u}_i = \frac{\mathbf{a}_i - \mathbf{p} - \mathbf{R}\,\mathbf{b}_i}{l_i} \tag{2.2}$$

The inverse kinematics of a CDPR is therefore explicit and inexpensive to evaluate, which is one of the reasons why these robots are attractive for real-time control [7, p. 121]. The forward problem, on the contrary, must be solved numerically from the m equations (2.1) [7, Sect. 4.2.2].

- **Static equilibrium and structure matrix.** Each cable pulls the platform towards its exit point with a tension $\tau_i > 0$, applying a force $\tau_i\,\mathbf{u}_i$ and a moment $(\mathbf{R}\,\mathbf{b}_i) \times \tau_i\,\mathbf{u}_i$. Writing the equilibrium of forces and moments gives

$$\mathbf{W}\,\boldsymbol{\tau} + \mathbf{w}_e = \mathbf{0} \tag{2.3}$$

where $\boldsymbol{\tau} = [\tau_1, \ldots, \tau_m]^\mathsf{T}$ is the vector of the m cable tensions, $\mathbf{w}_e$ is the external wrench applied to the platform — its weight and the payload — and $\mathbf{W}$, called the **structure matrix** or *wrench matrix*, gathers the geometry of the robot [15]:

$$\mathbf{W} = \begin{bmatrix} \mathbf{u}_1 & \cdots & \mathbf{u}_m \\ (\mathbf{R}\,\mathbf{b}_1) \times \mathbf{u}_1 & \cdots & (\mathbf{R}\,\mathbf{b}_m) \times \mathbf{u}_m \end{bmatrix} \in \mathbb{R}^{6\times m} \tag{2.4}$$

The upper block collects the force contributions and the lower block the moment contributions, three rows each: $\mathbf{W}$ therefore has six rows whatever the number n of degrees of freedom the robot actually controls. When the platform is constrained to fewer than six, only the n rows corresponding to the controlled directions are retained, and $\mathbf{W} \in \mathbb{R}^{n\times m}$.

Equation (2.3) is the central relation of the field. It shows that the platform can balance an external wrench only if the required tension vector exists *and* is admissible, that is

$$\tau_{min} \le \tau_i \le \tau_{max}, \qquad i = 1, \ldots, m \tag{2.5}$$

The lower bound $\tau_{min} > 0$ is the direct translation of the unilateral nature of the cables: it forbids a slack cable. The upper bound τ_{max} protects the cable and the structure.

- **Tension distribution.** When $m > n$ and $\mathbf{W}$ has full rank, its null space has dimension $m - n$, and equation (2.3) admits an infinity of solutions:

$$\boldsymbol{\tau} = \underbrace{-\,\mathbf{W}^{+}\,\mathbf{w}_e}_{\text{minimum-norm solution}} + \mathbf{N}\,\boldsymbol{\lambda} \tag{2.6}$$

where $\mathbf{W}^+$ is the pseudo-inverse of $\mathbf{W}$, $\mathbf{N}$ a basis of $\ker \mathbf{W}$ and $\boldsymbol{\lambda} \in \mathbb{R}^{m-n}$ a free parameter. The tension distribution problem consists in choosing $\boldsymbol{\lambda}$ at every instant so that all the components of $\boldsymbol{\tau}$ satisfy (2.5), which must be done in real time [16]. The conditioning of $\mathbf{W}$ is also an indicator of the quality of a given geometry: a poorly conditioned structure matrix means that large tension variations are needed to produce a small wrench, and that the corresponding degrees of freedom are badly controlled.

- **Cable elasticity.** Finally, the cable is not inextensible. Under a tension τ_i, a cable of free length l_i, of cross-section A and of Young's modulus E elongates by

$$\Delta l_i = \frac{\tau_i \, l_i}{E \, A} \tag{2.7}$$

This elongation, added to the sagging caused by the cable's own weight, quantifies the third limitation listed in Section 2.2.2: it is the difference between the length delivered by the winch and the effective distance to the platform [15].

2.2.4 Classification by Workspace

The workspace of a CDPR is defined as the region where the platform can move while maintaining positive tension in all cables. Two principal types of workspace exist based on the dimension of operation:

- **Planar workspace**: the platform moves in a two-dimensional plane, so at most three degrees of freedom are controlled. This configuration uses fewer cables and is suitable for tasks such as surface manipulation, cutting, and planar positioning.
- **Spatial workspace (3D)**: the platform moves in three-dimensional space. This configuration requires more cables and a more complex control, but enables full six-degrees-of-freedom positioning for tasks such as assembly, inspection, and large-scale manipulation.

2.2.5 Classification by Kinematic Configuration

CDPRs are classified based on the relationship between the number of cables (m) and the degrees of freedom of the moving platform (n). This classification determines the control complexity, stiffness, and application suitability, and it can be read directly on equation (2.3).

Fully Constrained CDPRs ($m = n + 1$)

In fully constrained CDPRs, the number of cables equals the degrees of freedom plus one (Figure 2.4a). The platform pose is fully determined by the cable lengths, and tension

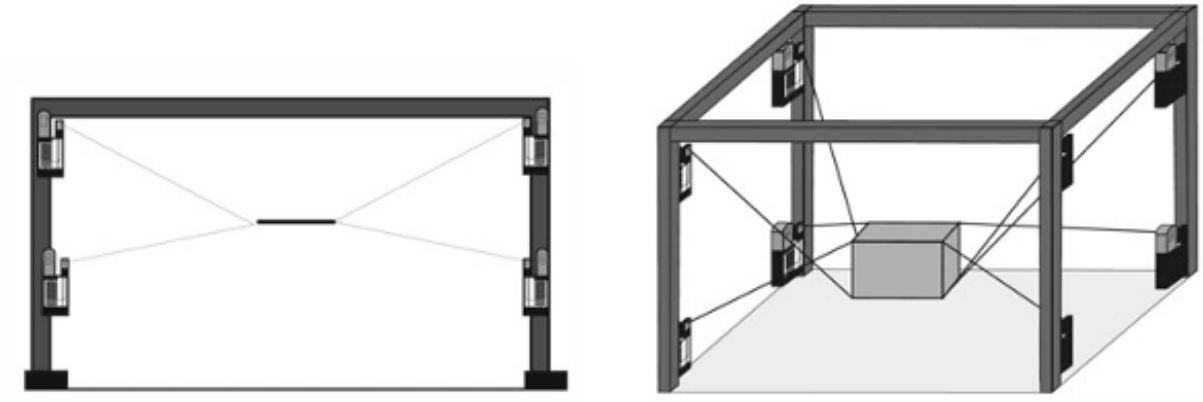
is maintained through the geometric arrangement of the cables combined with proper actuation, without relying on external forces such as gravity. This configuration offers a good balance between control complexity and achievable workspace, making it one of the most commonly used architectures in practical CDPR designs.

Redundantly Constrained CDPRs ***($m > n + 1$)***

Redundantly constrained CDPRs employ more cables than the minimum number required to fully constrain the platform (Figure 2.4b). The additional cables improve the stiffness, load capacity, and tension distribution of the system, allowing the robot to better withstand external forces and disturbances while handling larger payloads. However, this redundancy also increases the complexity of the control system, since the null space of $\mathbf{W}$ has dimension $m - n$ and the tension distribution problem of equation (2.6) admits multiple valid solutions that must be resolved in real time [16].

Under-Constrained CDPRs ***($m \leq n$)***

Under-constrained CDPRs use fewer cables than would be required to fully constrain the platform. In this configuration, gravity plays a major role in determining the equilibrium orientation and position of the platform, since the cables alone cannot control all degrees of freedom. This results in a simpler and lighter mechanical structure, but the system becomes more sensitive to external disturbances. Under-constrained architectures are commonly found in large-scale applications such as suspended camera systems or crane-like operations.



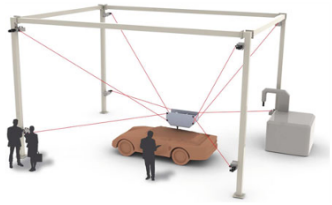
(a) Fully constrained (b) Redundantly constrained

Figure 2.4: Cable configurations of a CDPR. Adapted from [7].

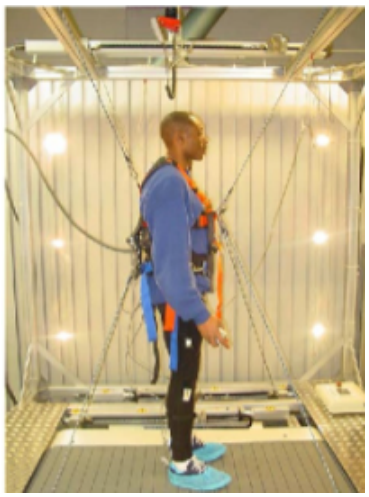
2.2.6 Applications of CDPRs

Cable-driven parallel robots have found applications in various domains due to their unique characteristics:

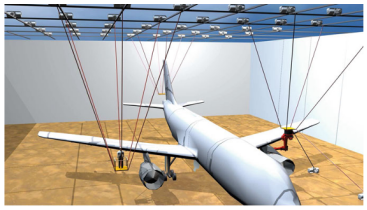
- **Industrial handling**: large-scale manipulation of heavy payloads in manufacturing and logistics (Figure 2.5a).
- **Medical rehabilitation**: cable-based systems provide adjustable resistance and support for patient limb training (Figure 2.5b).
- **Aircraft maintenance**: cable-driven systems enable large-scale access and positioning around aircraft structures for inspection and maintenance tasks (Figure 2.5c).



(a) Industrial handling



(b) Medical rehabilitation



(c) Aircraft maintenance

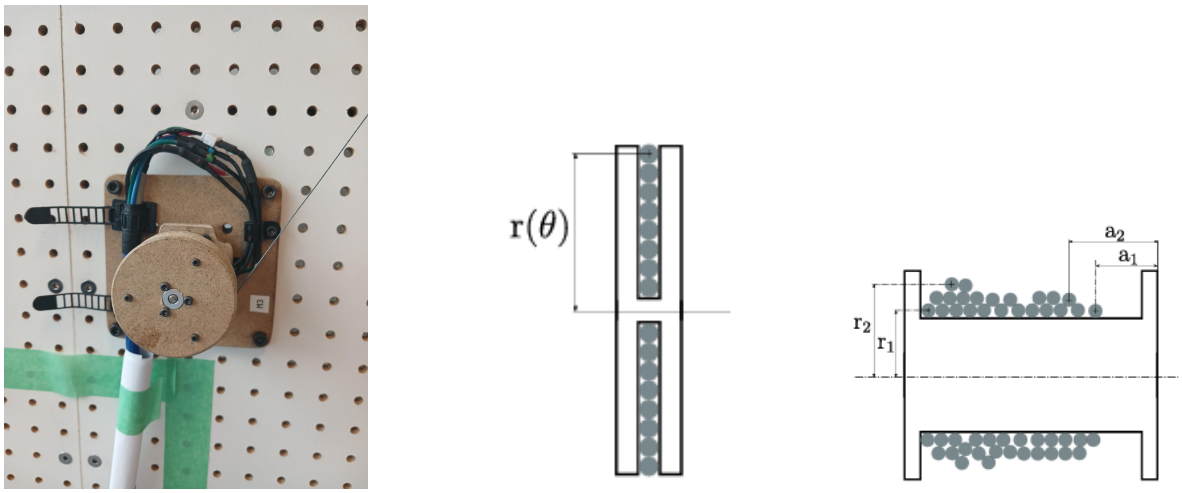
Figure 2.5: Examples of CDPR applications. Adapted from [7].

2.3 Cable Actuation Units

The cable actuation unit, or winch, is the component that converts the motor rotation into a cable displacement while supporting the tension. Its design directly determines the accuracy with which the cable length is known, and therefore the accuracy of the whole robot. Three families of architectures are reported in the literature [17].

2.3.1 Rotating Drum

In the rotating drum design, the drum performs a pure rotation around its axis and the cable is wound directly onto its surface. This is the simplest and most compact configuration, but the cable is not guided: successive wraps stack unevenly and may overlap. The effective winding radius then becomes a function of the rotation angle, so the relation between the motor rotation and the cable displacement is no longer linear, and the cable exit point moves along the drum axis.



(a) (b) (c)

Figure 2.6: Rotating drum configuration: (b) and (c) reproduced from [7]

2.3.2 Rototranslating Drum

In the rototranslating design, the drum both rotates and translates along its axis (Figure 2.7). A screw–nut mechanism converts the motor rotation into a synchronised axial displacement, so that the cable is laid turn after turn into a helical groove of constant radius and the cable exit point remains fixed with respect to the frame. The winding radius is therefore known and constant over the whole cable length, at the cost of a more complex mechanical assembly [17].

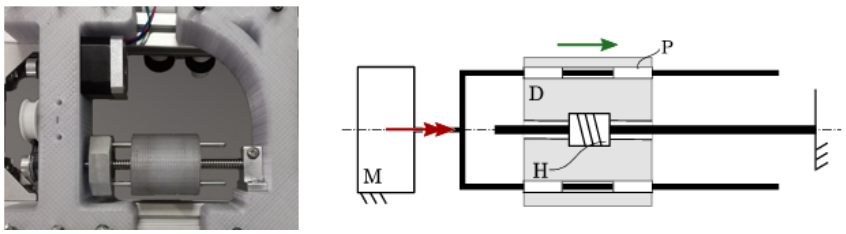


Figure 2.7: Rototranslating drum actuation unit. Adapted from [17].

2.3.3 Drum with Spooling Helper

The spooling helper uses an auxiliary mechanism to guide the cable onto a fixed drum (Figure 2.8). A moving guide eyelet or roller traverses back and forth along the drum width, ensuring an even distribution of the cable without requiring the drum itself to translate. This solution keeps the drum simple but adds a second actuated or synchronised mechanism and increases the overall volume of the unit [18].

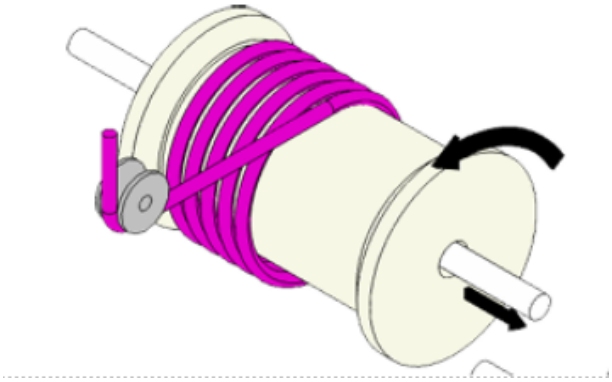


Figure 2.8: Drum with spooling helper. Adapted from [18].

2.3.4 Towards Self-Contained Winch Units

Most of the units reported in the literature are conceived as purely mechanical transmissions: the motor, its drive and, when present, the force sensor are distributed over the frame and connected to a central controller. A more recent trend consists in integrating the actuation, the sensing and the control electronics into a single compact module communicating over a field bus [17, 18]. Such a self-contained unit simplifies the wiring of a multi-winch robot, makes the architecture modular, and allows the tension to be regulated locally. This is the approach followed in the present work.

2.4 Control and Tension Sensing in CDPRs

The modelling of Section 2.2.3 shows that a cable length and a cable tension have to be mastered simultaneously. This section reviews how the literature controls these two quantities, and how the tension is obtained.

2.4.1 Control Strategies

Because a CDPR is actuated through flexible elements and is often redundant, its control cannot be reduced to the position control of independent axes. Three families of strategies are reported [7, 19]:

- **Length control**, in which each winch tracks the cable length computed by the inverse kinematics of equation (2.1). It is simple and fast, but it ignores the tension: any error in the cable model, or any deformation of the structure, produces either a slack cable or an overload. Compensating these modelling errors is an active research topic, addressed by adaptive and robust control schemes [20].
- **Tension control**, in which each winch regulates the force in its cable. It guarantees

that the cables stay inside the admissible interval of equation (2.5), but it cannot by itself impose a trajectory to the platform.

- **Hybrid position/force control**, in which the two previous strategies are combined: part of the cables is length-controlled to impose the trajectory, while the remaining ones are tension-controlled to keep the whole set inside its admissible interval [19]. It is today the reference approach for redundant CDPRs, and it requires each actuation unit to be able to measure the tension of its own cable [21].

2.4.2 Tension Sensing

Two families of solutions are used to obtain the cable tension [22]:

- **Indirect estimation** from the motor current or torque. It requires no additional sensor, but the estimation is corrupted by the friction of the transmission and by the efficiency of the reduction stage, which are difficult to model and vary with the operating point.
- **Direct measurement** with a force sensor placed on the cable path, typically a load cell supporting the guiding pulley. The measurement is independent of the transmission, at the cost of an additional sensor, of its calibration, and of an acquisition chain fast enough to be inserted in the control loop.

Within the direct measurement family, three concepts of integration of a one-axis force sensor into a cable robot are reported [7] and compared in Table 2.2.

Table 2.2: Concepts for integrating a one-axis force sensor into a cable robot. Adapted from [7].

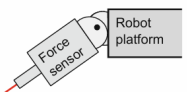
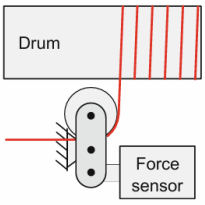
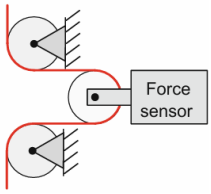
Concept	**Principle**	**Layout**
Platform integrated	The sensor is placed between the distal end of the cable and the moving platform, so it measures the cable force directly and is almost free of parasitic inertia and friction. Its drawback is that the signal and the power supply must be routed to the moving platform.	

Table 2.2 – continued from previous page

Concept	Principle	Layout
Winch integrated	The sensor supports a guiding pulley located inside the actuation unit, immediately after the drum. It remains fixed to the frame, which keeps the wiring simple, but it reads the resultant of the two cable strands rather than the tension itself.	
Pulley mechanism	The sensor is inserted in a deflection pulley mechanism placed on the cable path. It gives a linear estimate of the cable force under the assumption of a frictionless pulley, and can be located at the proximal end, where it does not move.	

▶ The winch-integrated concept was retained for the unit developed here, since the objective is a self-contained module: keeping the sensor inside the winch avoids any electrical connection to the moving platform, and makes the tension an independently regulated variable rather than a quantity inferred from the actuator. Its mechanical integration is detailed in Chapter 3.

Conclusion

Three decisions come out of this review. The unilateral nature of the cables imposes a strictly positive tension at every instant, so the winch cannot be a simple length actuator. Among the existing actuation units, only the rototranslating drum guarantees a constant winding radius and a fixed cable exit point with a single actuator. And hybrid position/force control, the reference approach for redundant CDPRs, requires each unit to measure its own cable tension, which a load cell placed on the guiding pulley provides without any connection to the moving platform.

These three choices define the winch designed in the following chapters.

Chapter 3

Design Study and Component Selection

Introduction

This chapter presents the design process of the self-contained winch unit, starting from the analysis of the existing setup, through the selection of the winch architecture and of the components, to the geometric definition of the unit. The methodology follows a top-down engineering approach: first defining the winch concept, then selecting the components, and finally refining the geometry that integrates them.

3.1 Analysis of the Existing Winch

An existing experimental setup, developed within the Mechatronics Laboratory under Dr. Mitchell Rushton, served as the starting point for this project. The setup includes a complete control system and consists of multiple winch units mounted on a 1/4-inch pegboard, forming a planar cable-driven parallel robot (Figure 3.1).

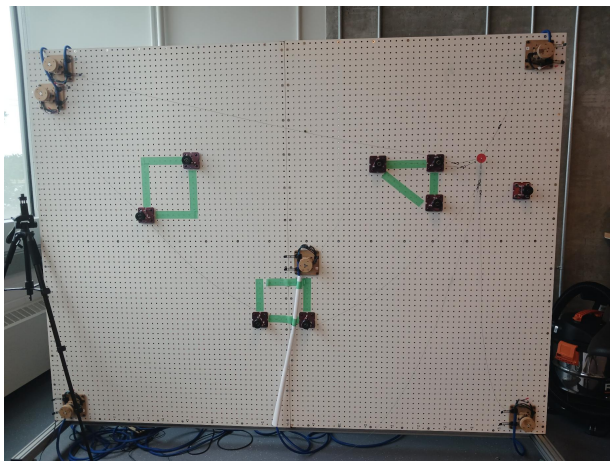


Figure 3.1: Planar CDPR with multiple winch units mounted on a 1/4-inch pegboard

Each winch unit in this setup relies on a **rotating drum** design, where the drum performs pure rotation to wind and unwind the cable directly onto its surface (Figure 3.2). However, the drum is **not grooved**. As a result, successive cable wraps stack unevenly, causing the effective winding radius to vary and introducing inaccuracies in the relation

between motor rotation and cable displacement. The unit also has no force sensor. The tension is estimated from the motor current, but this estimate is noisy, because the cogging torque of the BLDC motor adds a ripple of the same order as the torque to be measured.

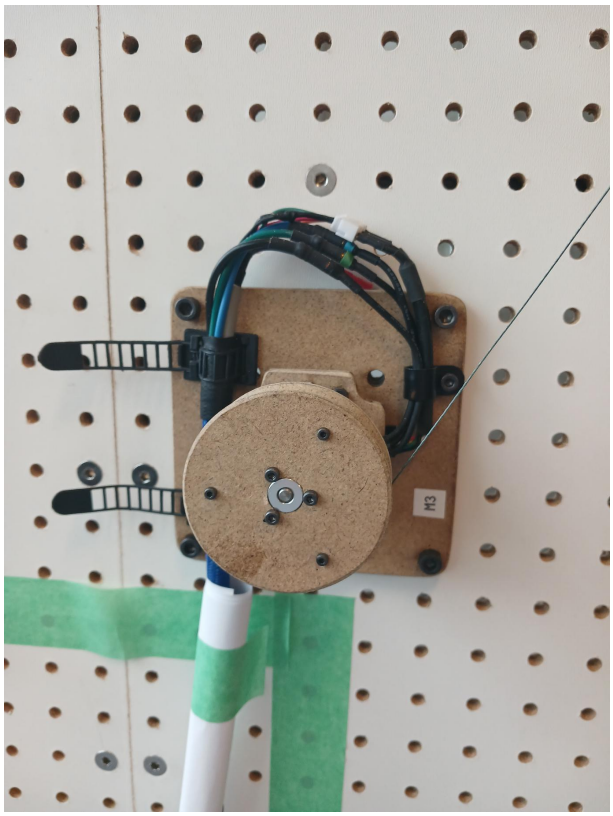


Figure 3.2: Winch unit of the existing setup

Building on the limitations identified above, the objectives for the new winch unit are defined as follows:

- Design a fully **self-contained winch unit**, integrating the guiding pulley, the actuation and the sensing electronics within a single compact module.
- Implement **active tension control** to ensure a stable and continuously monitored cable tension.
- Design a **grooved drum with a constant winding radius**, eliminating the variable radius and the uneven cable stacking observed in the existing rotating drum design.

3.2 Selection of the Winch Architecture

The actuator is the core component of a CDPR winch unit: it converts electrical energy into mechanical motion in order to control the cable length and tension. The three

architectures reported in the literature — rotating drum, rototranslating drum, and drum with spooling helper — were presented in Section 2.3. Their comparison with respect to the objectives defined above is summarised in Table 3.1.

Table 3.1: Comparison of the candidate winch architectures

Criterion	**Rotating drum**	**Rototranslating**	**Spooling helper**
Winding radius	Variable	**Constant**	Constant
Cable exit point	Moving	**Fixed**	Fixed
Mechanical complexity	**Low**	Medium	High
Additional mechanism	**None**	Screw–nut	Guide carriage
Compactness	**High**	Medium	Low

▶ The **rototranslating drum** architecture was selected. The rotating drum is discarded because its variable winding radius makes the cable length unknown, which is unacceptable when no external position sensor is available. The spooling helper solves the winding problem but adds a second synchronised mechanism, which increases the volume and the part count of a module that must be replicated eight times. The rototranslating design is the only one that guarantees a constant winding radius and a fixed cable exit point with a single actuator, and its extra complexity is limited to a screw–nut transmission and its linear guiding.

3.3 Component Selection

3.3.1 Actuator Selection

A direct-drive brushless DC (BLDC) motor was selected for its high torque density and its mechanical simplicity, eliminating the need for a gearbox and therefore removing backlash. Specifically, the **mj5208** brushless motor from mjbots Robotic Systems [1] was chosen for its compact form factor and its compatibility with the moteus motor controller used in this project.



Figure 3.3: mj5208 brushless motor [1]

Characteristic	**Value**
Peak torque	1.7 N·m
Mass (with wires)	193 g
Peak power	600 W
K_v	330
Maximum RPM	7500
Outer diameter	63 mm
Length	25 mm

Table 3.2: mj5208 motor specifications [1]

In direct drive the motor torque is transmitted to the cable without reduction, so the maximum tension that the unit can sustain is imposed only by the peak torque of the motor and by the winding radius of the drum:

$$\tau_{max} = \frac{\Gamma_{max}}{R} \tag{3.1}$$

where:

- τ_{max} : maximum cable tension the unit can produce [N];
- Γ_{max} : peak torque of the motor [N·m];
- R : winding radius of the drum [m].

With $\Gamma_{max} = 1.7$ N·m and the drum radius $R = 26$ mm, established in Chapter 4, the unit can produce:

$$\tau_{max} = \frac{1.7}{0.026} \approx 65 \text{ N} \tag{3.2}$$

This value is the physical capability of the motor at peak torque. The torque limit actually configured in the controller is lower, which caps the tension available in operation (Chapter 6).

3.3.2 Motor Controller

The motor is driven by a dedicated **moteus r4.11** controller [2]. It performs field-oriented control (FOC) of the three motor phases, and integrates on board the magnetic encoder reading, the position and velocity loop and the current loop, both executed at the switching frequency, 30 kHz by default.

This matters for the present work: the fast inner loops are handled locally by the controller, so the host computer only has to close the outer tension loop, at a much lower rate. Its characteristics are given in Table 3.3.

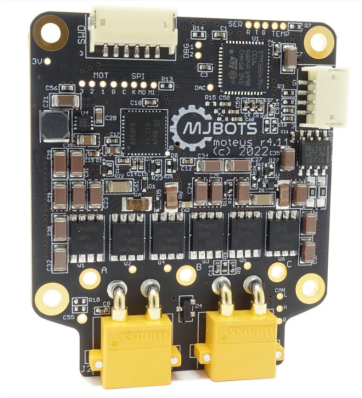


Figure 3.4: moteus r4.11 motor controller [2]

Characteristic	Value
Control type	3-phase brushless FOC
Voltage input	10–44 V ($\leq$ 10S)
Temperature range	-40 to 85°C
Peak phase current	100 A
Continuous phase current	12 A / 32 A (w/o and w/ thermal mgmt.)
Peak electrical power	900 W @ 30 V
Mass	14.2 g
Control rate	30 kHz (default)
PWM switching rate	15–60 kHz
Microprocessor	170 MHz 32-bit STM32G4
Max electrical frequency	2 kHz

Table 3.3: moteus r4.11 controller specifications [2]

3.3.3 Communication Interface

CAN-FD (*Controller Area Network with Flexible Data-rate*) is a serial bus standardised in ISO 11898 [23], in which all the nodes are connected to the same differential pair of wires. Each node broadcasts frames carrying an identifier rather than a destination address, and access to the bus is resolved by priority arbitration on that identifier, so no collision destroys a frame. In this work the link between the host computer and the motor controller is established through an **mjcanfd-usb-1x** adapter [3], operating at 1 Mbit/s in the arbitration phase and 5 Mbit/s in the data phase [24].

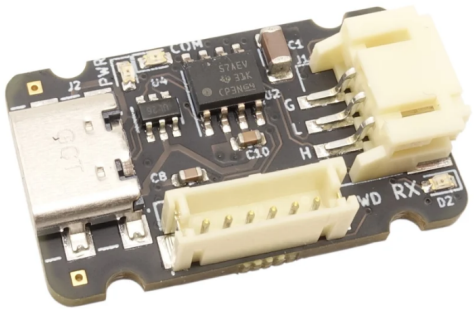


Figure 3.5: mjcanfd-usb-1x CAN-FD to USB adapter [3]

3.3.4 Tension Sensor Selection

As shown in Section 2.4, the cable tension can either be estimated from the motor current or measured directly by a force sensor. The direct measurement was retained here, since

the objective is to make the tension an independently regulated variable rather than a quantity inferred from the actuator.

Several technologies exist to measure a force:

- **Piezoelectric sensors**, which generate an electric charge when a crystal is placed under stress.
- **Force-sensing resistors**, whose polymer film changes resistance with the applied pressure.
- **Capacitive sensors**, in which the deflection of an electrode changes a capacitance.
- **Strain gauge load cells**, in which gauges bonded on a deformable metallic body and connected as a Wheatstone bridge convert the deformation into a voltage.

▶ The **strain gauge load cell** was retained. The piezoelectric sensor responds to a variation of force and not to its absolute value, so it cannot hold a static measurement. The force-sensing resistor is too imprecise to close a control loop, and the capacitive sensor is more sensitive to temperature and less available off the shelf.

Measurement principle

A load cell is a force sensor built from strain gauges bonded onto a deformable metallic body and connected as a Wheatstone bridge [8]. The force applied to the body deforms it, which changes the cross-section of the gauge wires and therefore their electrical resistance, unbalancing the bridge (Figure 3.6). The bridge then delivers a differential voltage proportional to the applied force, of the order of a few millivolts per volt of excitation [4]. This signal is far too small to be read directly by the analogue input of a microcontroller, so a dedicated instrumentation amplifier and analogue-to-digital converter are required; the HX711 used in this project is presented in Section 3.3.5.

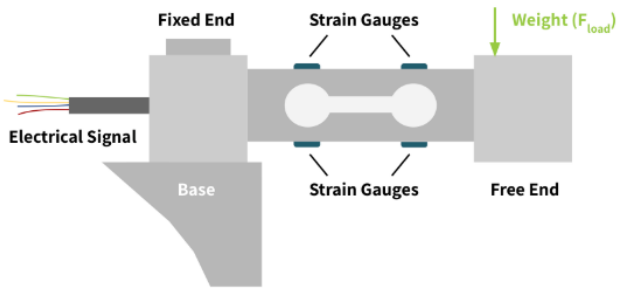
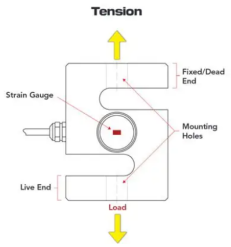
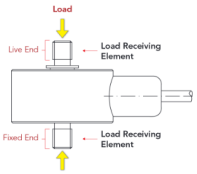
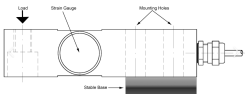
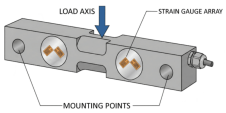


Figure 3.6: Wheatstone bridge of a strain gauge load cell. Adapted from [8].

Load cell geometries

Strain gauge load cells differ by the shape of their deformable body, which imposes how the sensor is mounted and how the force reaches it. The four usual geometries are compared in Table 3.4.

Table 3.4: Strain gauge load cell geometries. Adapted from [8].

Geometry	**Mounting**	**Layout**
S-beam	Inserted in series in the cable itself, loaded in tension between its two threaded ends.	
Button / compression	Loaded in compression only, between two flat faces.	
Shear beam	Bolted at one end, the load being applied at the other end.	
Bending beam	Clamped at one end on a flat surface, the load being applied at the free end.	

► The **bending beam** geometry was retained. The S-beam has to be inserted in the cable itself, which adds a moving mass and a mechanical weak point on the cable path. The button cell only takes a compressive load and requires a perfectly aligned force path. The shear beam is designed for capacities far above the range needed here. The bending beam is the only one that mounts flat on the base plate, with the pulley bracket fixed directly on its free end.

The **CZL601** miniature load cell was selected accordingly; its characteristics are given in Table 3.5.

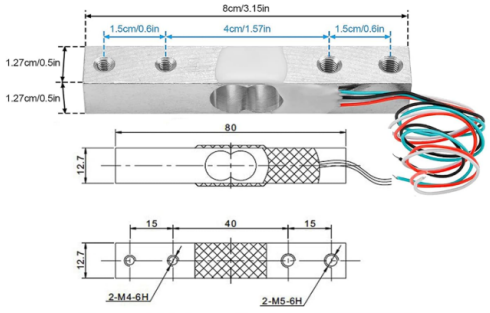


Figure 3.7: CZL601 miniature load cell [4]

Characteristic	**Value**
Rated capacity	0–10 kg (≈ 98 N)
Accuracy	0.02%–0.03% F.S. (C2 or C3 class)
Excitation voltage	5 V
Material	Standard aluminium alloy
Water protection	IP65
Temperature range	−20°C to 65°C
Temperature drift	±0.02%–0.03% / 10°C

Table 3.5: CZL601 load cell specifications [4]

3.3.5 Signal Conditioning

The differential voltage delivered by the load cell is of the order of a few millivolts per volt of excitation, and cannot be read by the analogue input of a microcontroller. A dedicated conditioning stage is therefore required between the sensor and the acquisition board.

The **HX711** was selected for this function (Figure 3.8). It is a 24-bit sigma-delta analogue-to-digital converter designed specifically for bridge sensors [9]. Its characteristics are given in Table 3.6.

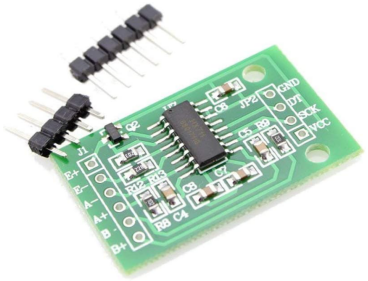


Figure 3.8: HX711 24-bit analogue-to-digital converter module

Table 3.6: HX711 specifications [9]

Characteristic	**Value**
Resolution	24 bits
Channel A gain	128 or 64 (±20 mV or ±40 mV full scale)
Channel B gain	32 (±80 mV full scale)
Output data rate	10 or 80 samples per second
Supply voltage	2.6–5.5 V
Bridge excitation	Provided by the on-chip regulator
Interface	Two-wire serial (DOUT, PD_SCK)

The output data rate is the parameter that couples this component to the control: the tension loop of Chapter 6 runs at 50 Hz, so the converter should be strapped to its 80 SPS mode. The module used in this work operates at 10 SPS, which makes the feedback five times slower than the loop that uses it; this limitation is quantified in Chapter 6 and listed among the perspectives of the general conclusion.

3.3.6 Microcontroller Selection

An Arduino Mega was selected to interface with the load cell and to handle the real-time acquisition of the tension. It was retained for its compatibility with the HX711 amplifier, its hardware serial ports allowing a continuous transmission to the host computer without disturbing the acquisition, and its availability in the laboratory.



Figure 3.9: Arduino Mega board

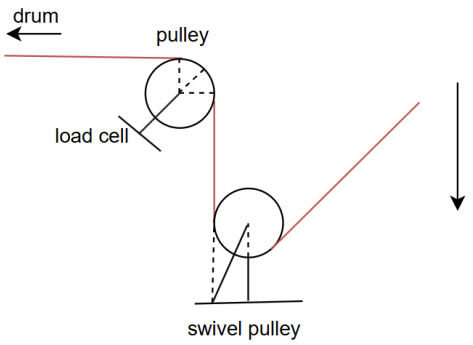
3.4 Geometric Design

3.4.1 Pulley Geometry Selection

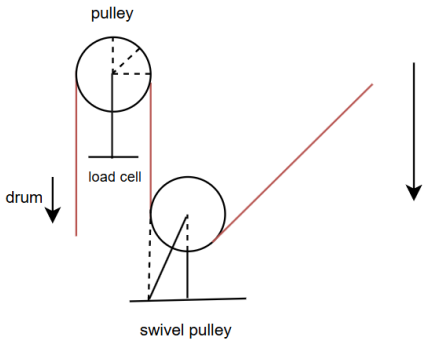
Two candidate geometries were considered for positioning the load cell relative to the drum and to the swivel pulley (Figure 3.10).

In **configuration (a)** (Figure 3.10a), the cable exits the drum horizontally and passes over a small fixed pulley mounted directly on the load cell, before being redirected downward to the swivel pulley and then up toward the platform. This arrangement keeps the load cell close to the drum axis, resulting in a compact and direct force path with a single redirection before reaching the swivel pulley.

In **configuration (b)** (Figure 3.10b), the cable exits the drum vertically and passes directly through a pulley mounted above the load cell before reaching the swivel pulley. This arrangement also allows an accurate measurement, but it results in a taller overall assembly and requires the drum to be oriented vertically relative to the pulley axis.



(a)



(b)

Figure 3.10: Candidate pulley geometries for the integration of the load cell

In both cases the load cell does not read the cable tension directly. The pulley transmits to its axis the resultant of the two cable strands, carried by their bisector, whose magnitude depends on the angle α between them:

$$F_{meas} = 2\,T\cos\left(\frac{\alpha}{2}\right) \tag{3.3}$$

where:

- F_{meas} : force applied to the load cell [N];
- T : tension in the cable [N];
- α : angle between the incoming and the outgoing strand [°].

The two strands are perpendicular in configuration (a) and parallel in configuration (b), which gives

$$F_{meas}^{(a)} = 2\,T\cos 45° = T\sqrt{2} \approx 1.414\,T, \qquad F_{meas}^{(b)} = 2\,T\cos 0° = 2\,T \tag{3.4}$$

▶ Configuration (a) was selected. It gives a more compact overall geometry and a more direct force path between the drum, the load cell and the swivel pulley.

3.4.2 Overall Winch Layout

Figure 3.11 shows the overall geometric layout of the winch unit, integrating the motor, the drum and the load cell within a compact, self-contained envelope. The motor is mounted coaxially with the drum, so that no transmission element is inserted between them. The load cell is fixed on the same base plate, immediately at the cable exit, which keeps the force path short and avoids introducing additional compliance between the cable and the sensor. The swivel pulley is placed at the output of the unit, where the cable leaves towards the mobile platform.

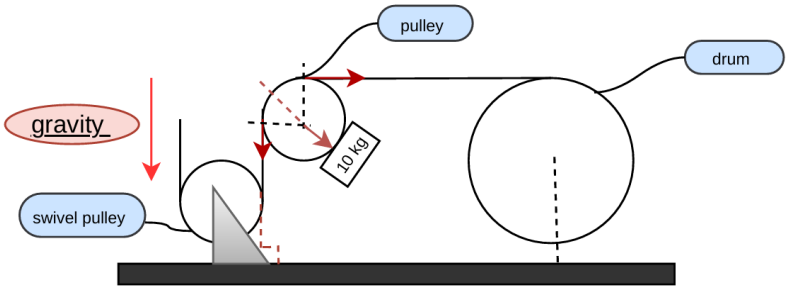


Figure 3.11: Overall geometric layout of the winch unit

Conclusion

Every component of the winch is now fixed, and each choice constrains the next: the rototranslating architecture calls for a screw–nut transmission, the direct-drive mj5208

motor sets the maximum tension of the unit at 65 N, and the winch-integrated sensing configuration imposes a bending beam load cell with its HX711 conditioning stage.

The next chapter turns these choices into a machine: the geometry of the drum, of the two pulleys and of the base plate, and the standard components that drive them.

Chapter

4

Mechanical Design

Introduction

This chapter presents the mechanical design of the winch unit. It begins with an overview of the complete assembly, then details each element in turn: the grooved drum, the two pulleys, the load cell support and the base plate, followed by the standard transmission components that drive them. It closes with the integration of eight identical units on the target robot.

4.1 Overview of the Winch Unit

The complete unit is shown in Figure 4.1, in section in Figure 4.2, and with its elements identified in Figure 4.3. Each of them is detailed in the sections that follow. Particular attention was paid to the stack-up of tolerances to ensure smooth rotation of all moving parts.

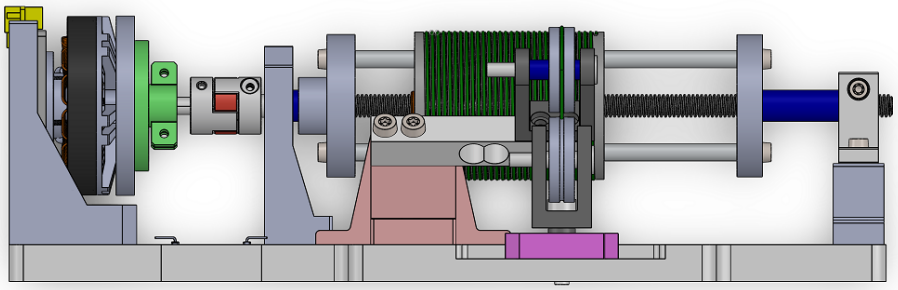


Figure 4.1: Full design of the self-contained winch unit

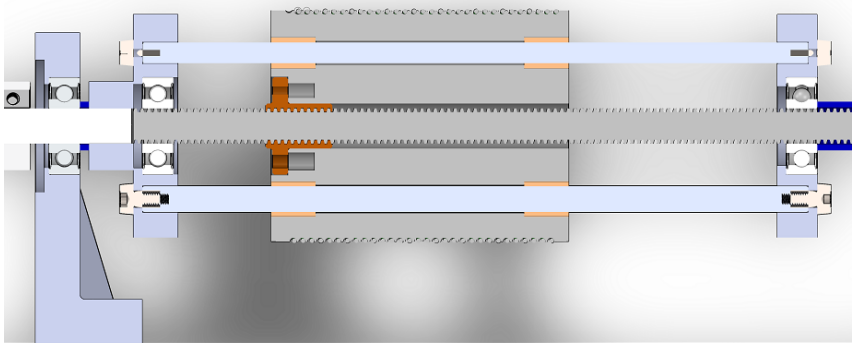


Figure 4.2: Section view of the winch unit

The section view of Figure 4.2 shows the rototranslating mechanism: the lead screw is grounded to the base plate by the SK8 supports, while the motor drives the cover, the drum and the nut embedded in it as a single rotating body. The screw–nut pair therefore acts as a helical joint between the frame and the drum, so the single rotational input of the motor produces one revolution and one lead of 2 mm of axial travel simultaneously.

A single actuator therefore produces both motions, and the ratio between them is imposed by the geometry alone: the drum advances by one groove pitch for each turn of cable wound.

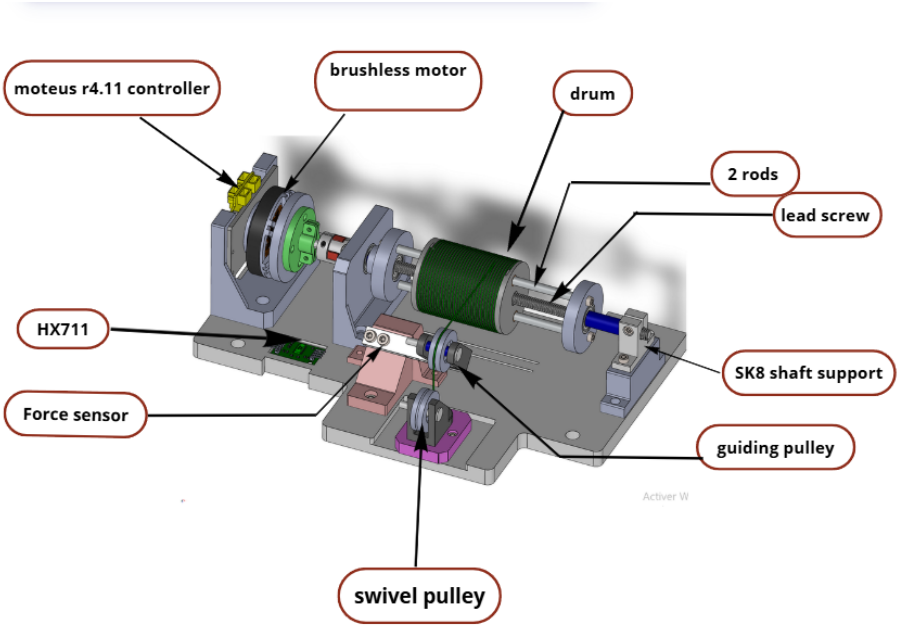


Figure 4.3: Components of the winch unit

4.2 Drum Design

4.2.1 Helical Groove Geometry

To ensure a constant winding radius and prevent cable overlapping, a helical groove is machined into the drum surface (Figure 4.4). Its pitch P was set to 2 mm in order to synchronise the rotation of the drum with the axial translation imposed by the lead screw: the screw selected in Section 4.6 has a lead of 2 mm, and matching the two values guarantees that the cable is guided into the centre of the groove over the whole 5 m of cable.

The profile of the groove follows the usual design rules for wire rope drums [28]. The groove radius is taken slightly larger than the radius of the cable, so that the cable seats in the groove without being pinched, and the depth is chosen so that the cable is held laterally without rubbing on the crests.

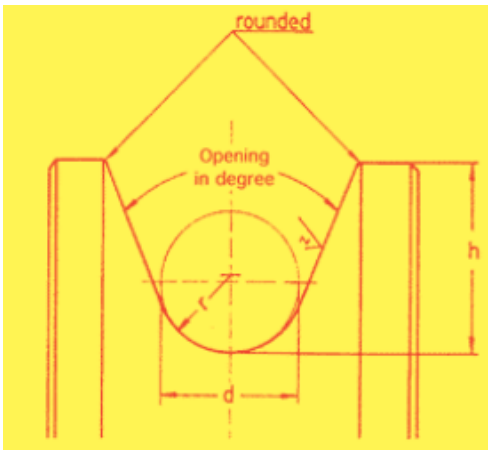


Figure 4.4: Profile of the helical groove machined on the drum

4.2.2 Drum Dimensioning

The drum stores the whole cable in a single layer, so its diameter and its length are fixed by three constraints: the minimum bending radius of the cable, the total length to be stored, and the pitch of the helical groove.

Design parameters

- Total cable length (L): 5000 mm
- Cable diameter (d): 1 mm
- Drum diameter (D): 52 mm
- Helical groove pitch (P): 2 mm

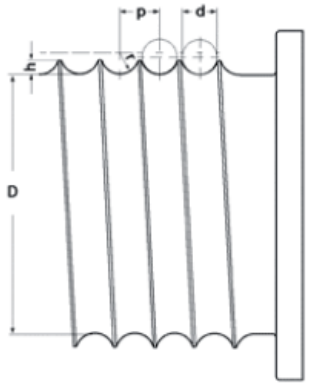


Figure 4.5: Drum and groove parameters

Symbol	Parameter	Value
D	Drum diameter	52 mm
d	Cable diameter	1 mm
P	Groove pitch	2 mm
r	Groove radius	0.75 mm

Table 4.1: Drum and groove parameters

Choice of the drum diameter

The diameter must first satisfy the minimum bending ratio recommended for wire rope drums, $D/d \geq 18$ [28]. A cable wound on a drum smaller than this limit is subjected to a bending stress that shortens its fatigue life. With $d = 1$ mm:

$$\frac{D}{d} = \frac{52}{1} = 52 \ \geq \ 18 \tag{4.1}$$

Number of turns

The number of turns required to store the whole cable is given by the circumference of the drum:

$$N = \frac{L}{\pi D} = \frac{5000}{\pi \times 52} = 30.6 \ \rightarrow \ 31 \text{ turns} \tag{4.2}$$

Winding length

The axial length occupied by the cable follows from the number of turns and the pitch of the groove:

$$W = N \cdot P = 31 \times 2 = 62 \text{ mm} \tag{4.3}$$

Resulting performance

Two quantities of the drum directly condition the control presented in Chapter 6. The maximum tension the unit can produce follows from the peak torque of the motor and the winding radius $R = D/2 = 26$ mm:

$$\tau_{max} = \frac{\Gamma_{max}}{R} = \frac{1.7}{0.026} \approx 65 \text{ N} \tag{4.4}$$

This value is the physical capability of the motor at peak torque. The torque limit actually configured in the controller is lower, which caps the tension available in operation (Chapter 6).

The maximum cable speed follows from the velocity saturation of $\omega_{max} = 0.9$ rev/s applied by the controller:

$$v_{max} = \pi D\, \omega_{max} = \pi \times 0.052 \times 0.9 \approx 0.147 \text{ m/s} \tag{4.5}$$

Table 4.2: Summary of the drum dimensioning

Quantity	**Symbol**	**Value**
Drum diameter	D	52 mm
Bending ratio	D/d	52
Number of turns	N	31
Winding length	W	62 mm
Maximum cable tension	τ_{max}	65 N
Maximum cable speed	v_{max}	0.147 m/s

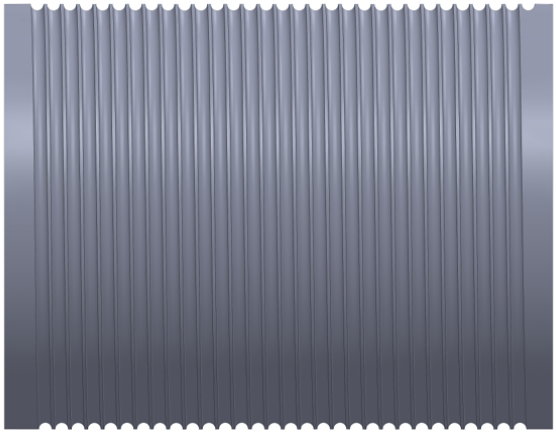


Figure 4.6: CAD model of the grooved drum

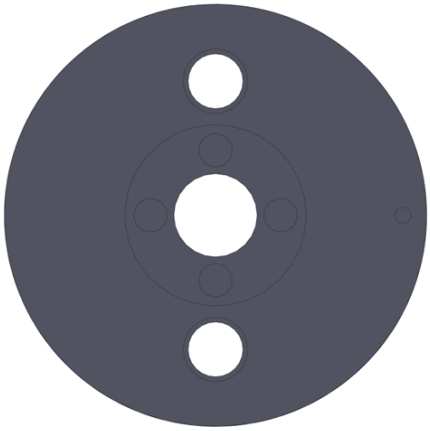


Figure 4.7: Section view of the grooved drum

4.3 Pulley Design

The cable follows a fixed path inside the unit: it leaves the drum, is deflected by the **measuring pulley** mounted on the load cell, and then reaches the **swivel pulley** through which it exits towards the platform. The two pulleys answer different requirements and are therefore designed differently.

4.3.1 Measuring Pulley

The measuring pulley deflects the cable at a constant wrap angle, so that the resultant of its two strands loads the sensing axis of the load cell. It has a single degree of freedom, rotation about its own axis. It runs on a ball bearing so that it rolls without sliding.

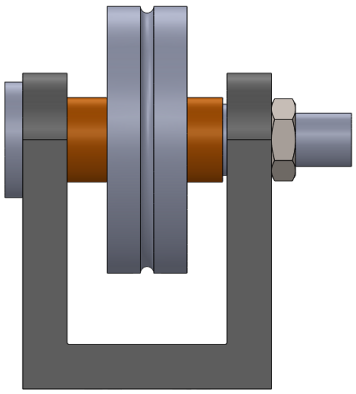


Figure 4.8: The measuring pulley design

4.3.2 Swivel Pulley

A fixed pulley would introduce lateral friction as the cable angle changes with the platform movement, so a **dual-axis swivel pulley** was designed instead (Figure 4.9). This pulley provides **two degrees of freedom**: a horizontal rotation axis for the cable rolling, and a vertical pivot axis that allows the whole assembly to swivel and self-align with the instantaneous direction of the cable as the platform moves.

A key feature of the design is the **hollow vertical axis**, which allows the cable to be routed through the centre of the pivot. Two reasons impose this coaxial guidance:

- **Invariant cable exit point.** The inverse kinematics of equation (2.1) assumes the exit point $\mathbf{a}_i$ to be fixed. The pivot axis is the only line of the assembly that does not move when the assembly swivels, so routing the cable along it keeps the exit point at the intersection of that axis with the pulley plane, whatever the swivel angle. An offset would make $\mathbf{a}_i$ describe a circle as the platform moves, and therefore depend on the platform pose — the *pulley kinematics* problem, usually compensated by a correction added to the model [7], and removed here mechanically.
- **No parasitic torque on the pivot.** A force whose line of action passes through an axis produces no moment about it. With the cable on the pivot axis, the incoming strand applies no moment whatever the tension, and the orientation of the assembly is imposed by the outgoing strand alone.

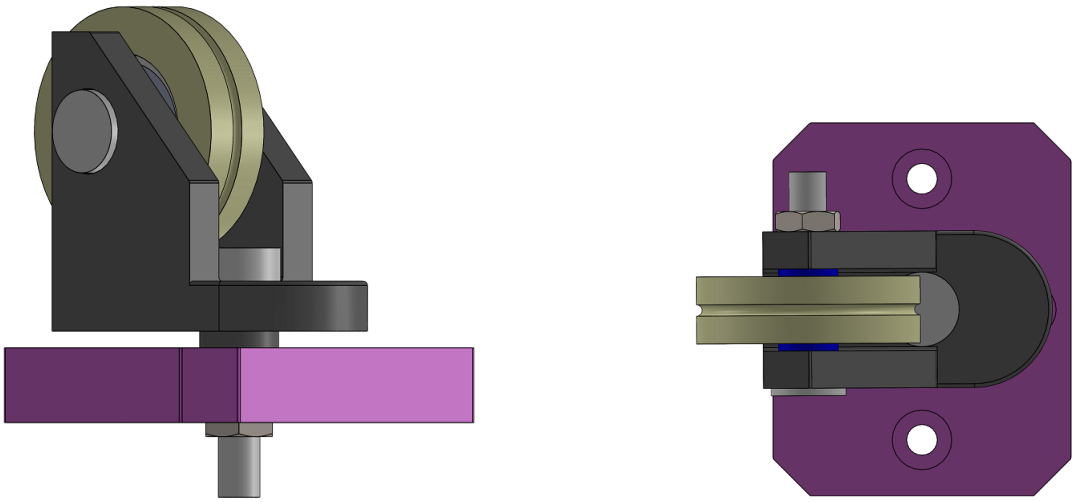


Figure 4.9: Swivel pulley assembly

4.4 Load Cell Support

A bending beam load cell measures the force applied along its sensing axis only: any component perpendicular to that axis is not measured and loads the beam in bending. The bracket must therefore align the sensing axis with the resultant of the cable tension.

In configuration (a) the two strands are perpendicular, so the resultant given by equation (3.3) has the magnitude $T\sqrt{2}$ and is carried by their bisector, at 45° from the horizontal. A dedicated bracket therefore holds the cell at 45°.

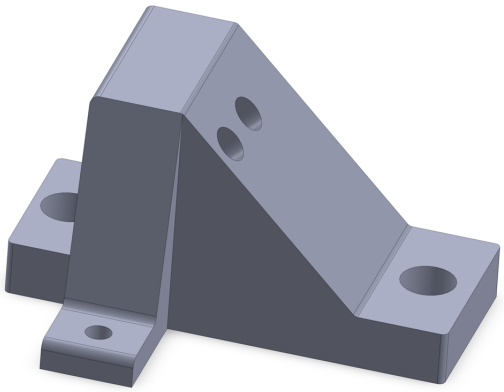


Figure 4.10: Load cell support holding the sensor at 45°

4.5 Base Plate

The base plate is the reference part of the unit: every subassembly — the motor mount, the SK8 supports of the lead screw, the load cell bracket and the swivel pulley — is bolted to its upper face, so all the alignments of the winch derive from a single datum (Figure 4.11). Its main features are:

- **Thickness of** 13 **mm**, set by the depth of the pockets rather than by the load.
- **Recessed housing for the swivel pulley**, which sets the vertical offset between the swivel and the measuring pulley and prevents the two from interfering, their planes being close to each other.
- **Pocket for the load cell and its bracket**, bringing the sensing axis to the height of the cable path without an additional support.
- **Clearance holes for** 1/4**" screws**, aligned with the hole pattern of the pegboard.

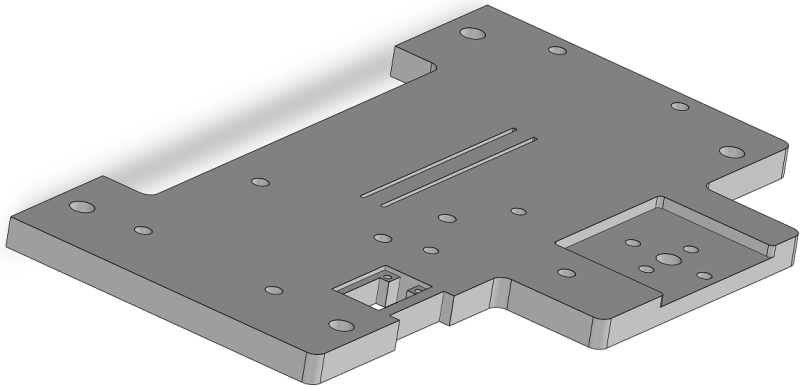


Figure 4.11: Base plate of the winch unit

4.6 Mechanical Transmission Components

The rototranslating architecture selected in Section 3.2 requires the drum to rotate and to translate simultaneously along its axis. This motion is obtained by combining a screw–nut transmission, a linear guiding system, rotational bearings and supports that hold the lead screw. These components are described below.

Lead screw

The conversion of the rotation into an axial translation is performed by a **lead screw**. Its lead is the parameter that couples the two motions: for one revolution of the drum, the assembly translates by exactly one lead. A lead of 2 mm was selected so that it matches the pitch of the helical groove machined on the drum, which guarantees that each new cable turn is laid precisely into the next groove:

$$P_{screw} = P_{groove} = 2 \text{ mm/rev} \tag{4.6}$$



Figure 4.12: Tr8×2 trapezoidal lead screw [5]

SK8 shaft supports

The lead screw is held at both ends by **SK8 shaft support blocks**, a standard aluminium component designed for 8 mm shafts (Figure 4.13) [26]. Their function is to clamp the screw rigidly to the base plate so that it *cannot rotate*: the nut, which is embedded in the drum, therefore screws itself along the fixed screw as the drum turns, and this is what converts the rotation of the motor into the axial translation of the assembly.



Figure 4.13: SK8 aluminium shaft support block for 8 mm shafts [26]

Guide rods and linear bushings

The screw being clamped to the base plate, the drum and its nut both rotate and advance along it. The motor, rigidly coupled to the drum, therefore travels with it, and its mount must be free to slide axially while being held against rotation. This is the role of the two **parallel guide rods** of 6 mm diameter on which the mount is carried.



Figure 4.14: Linear guide rod ∅6 × 200 mm [6]

The rods are used with **linear bushings**, which provide a low and, above all, constant friction along the stroke. This point is important for this application: since the tension is measured on the cable path and regulated in closed loop, any variable friction in the guiding would appear to the controller as a disturbance on the tension.

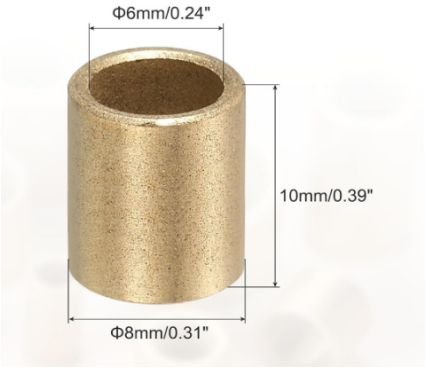


Figure 4.15: Self-lubricating cast brass sleeve bushing $6 \times 8 \times 10$ mm [25]

Ball bearings

Ball bearings support the rotating parts of the assembly. Rolling bearings were preferred to plain bearings because their friction torque is low and repeatable: a plain bearing would add a friction that varies with load and with time, and that the tension loop would have to reject. Two sizes of deep groove ball bearings are used, given in Table 4.3.

Table 4.3: Ball bearings used in the winch unit

Reference	**Bore**	**Outer dia.**	**Width**
626ZZ	6 mm	19 mm	6 mm
608ZZ	8 mm	22 mm	7 mm

Shaft coupler

A **flexible coupling** of the clamping type connects the 6 mm motor shaft to the 8 mm drum shaft (Figure 4.16) [27]. Two properties motivated this choice. Its clamping ends transmit the torque by friction over the whole circumference of the shaft, without the angular backlash that a set-screw or a jaw coupling would introduce; and its flexibility absorbs the residual angular and parallel misalignment between the two shafts.



Figure 4.16: Flexible aluminium shaft coupling, 6 mm to 8 mm [27]

4.7 Integration on the Target Robot

The unit described in this chapter is a single actuator of a larger machine. The target robot is built on a pegboard frame of 2450 × 1830 mm on which eight identical units are bolted, as shown in Figure 4.17. Each unit is fixed to the frame by 1/4" screws passing through the clearance holes of its base plate (Section 4.5), which are aligned with the hole pattern of the pegboard.

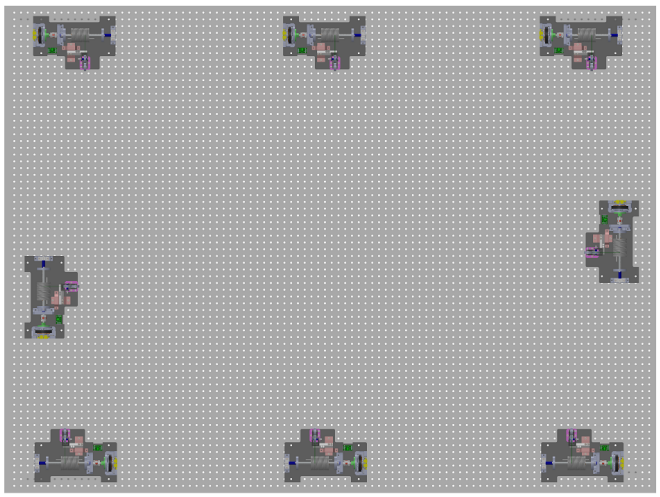


Figure 4.17: Target robot: the eight self-contained winch units bolted with 1/4" screws on the 2450 × 1830 mm pegboard frame.

Conclusion

This chapter has defined the complete mechanical unit: a grooved rototranslating drum driven through a fixed screw–nut pair, a measuring pulley that deflects the cable at a constant angle onto the load cell, and a swivel pulley that keeps the cable exit point fixed whatever the direction of the platform. Together, these three elements make the cable length and the cable tension two quantities that the geometry alone renders measurable.

The next chapter presents the electrical setup and the calibration that turn these measurable quantities into usable signals.

Chapter 5

Prototyping, Electrical Integration and Calibration

Introduction

The previous chapter established the mechanical design of the winch. This one turns that design into a working unit and prepares the bench on which it is operated. The manufacturing and the assembly of the prototype are presented first, then the two electrical chains of the system: the one that carries the commands from the computer to the motor, and the one that brings the measured cable tension back. Each chain is wired, checked and calibrated in turn, so that its behaviour is known before any closed-loop control is attempted.

5.1 Prototyping and Assembly

5.1.1 3D Printing

All the parts specific to the winch — the grooved drum, its cover, the load cell bracket, the pulley supports and the base plate — were produced by **fused deposition modelling (FDM)**, in which a thermoplastic filament is extruded layer by layer.

- **Material: PLA** was used for the prototype: it prints easily and holds its dimensions well, which is enough to validate the geometry and the assembly. **Polycarbonate** is retained for the final version.
- **Printer: Prusa CORE One** (Figure 5.1). Its enclosed chamber is what makes the polycarbonate version possible: this material warps and delaminates when printed in open air, and requires a stable ambient temperature around the part.



Figure 5.1: Prusa CORE One 3D printer

5.1.2 Assembled Prototype

The printed parts were assembled with the standard components selected in Chapter 3. The assembled unit is shown in Figure 5.2.

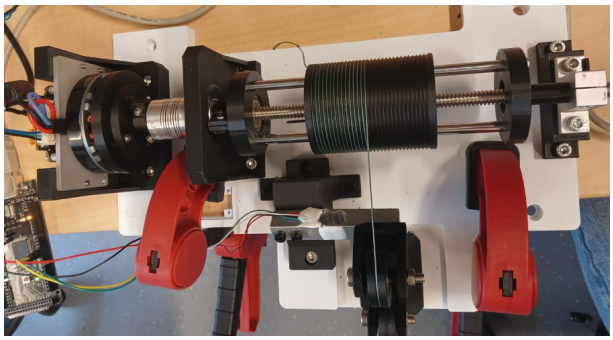


Figure 5.2: Assembled prototype of the self-contained winch unit

The prototype confirms three points of the design:

- **Single-layer winding.** The cable lies in one layer over the whole grooved length, each turn seating in its own groove, which validates the winding geometry computed in Section 4.2.2.
- **Synchronised translation.** The drum advances along its axis as it turns, so the cable always leaves the unit at the same point.
- **Compactness.** Every function of the unit actuation, cable spool , guidance and tension sensing is held within a single envelope,which meets the objective of a self-contained module set in Chapter 1.

5.2 Motor Actuation Chain

This section follows the command path from the computer down to the drum: the actuator and its controller, the way the controller drives the motor, the bus that carries

the commands, the interface with the host computer, and finally the diagnostic and the calibration that make the chain usable.

5.2.1 Control Structure of the Controller

The whole motor control is executed by the firmware of the moteus board: the host only writes a set-point, and everything between that set-point and the motor phases is computed on the board. The control is organised in two nested stages, both executed at the switching frequency, 30 kHz by default [29]. The complete structure is given in Figure 5.3.

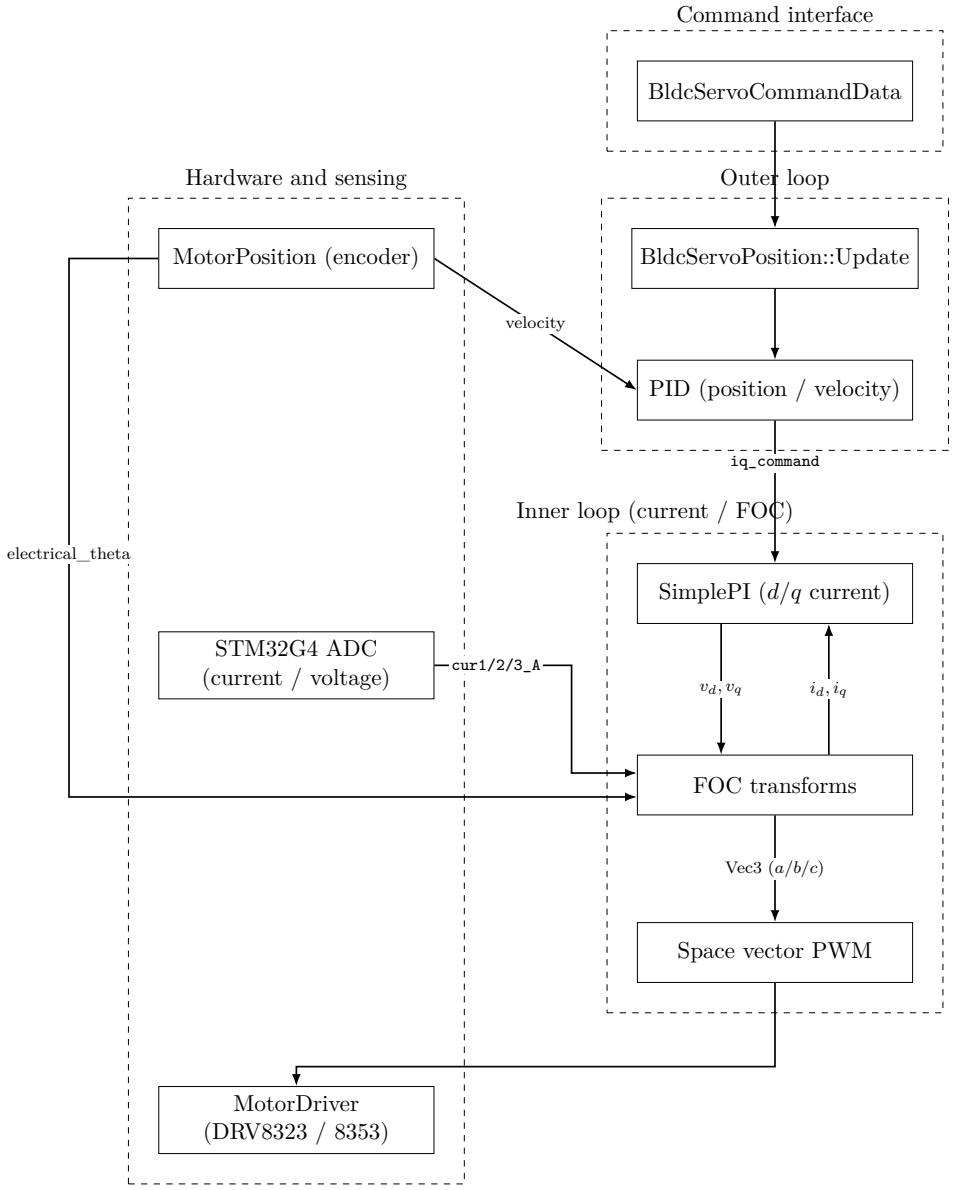


Figure 5.3: Control structure of the moteus firmware, adapted from [29]

- **Command interface:** the set-point received over CAN-FD is decoded into a command structure holding the target position, the velocity, the maximum torque and

the gain scales.

- **Outer loop:** a single integrated PID acting on the position and the velocity errors, whose output is a **current command** i_q^* proportional to the requested torque [29].
- **Inner loop:** a PI controller regulating the currents in the d–q frame; the FOC transforms map these quantities onto the three phases using the electrical angle of the rotor, and the space vector modulation converts them into the duty cycles of the three half-bridges [29].
- **Hardware and sensing:** the magnetic encoder supplies the velocity to the outer loop and the electrical angle to the transforms, the internal ADC measures the phase currents, and the gate driver applies the duty cycles to the motor.

The same command structure gives every control mode, depending on which fields are used [10].

Table 5.1: Control modes of the moteus controller [10]

Mode	**How it is obtained**
Position	A target position is given; both loops are active
Velocity	The position field is set to `NaN`: no position is imposed, and the controller tracks the commanded velocity
Torque	The PID gains are scaled to zero, leaving only the feedforward torque
Trajectory	A velocity limit and an acceleration limit are added, and the controller generates a constant-acceleration profile towards the target

5.2.2 Wiring and Interface with the Host Computer

The moteus board carries every connection of the actuation chain on a single printed circuit, which is what allows the electronics to remain inside the winch envelope. Its connectors are given in Figure 5.4.

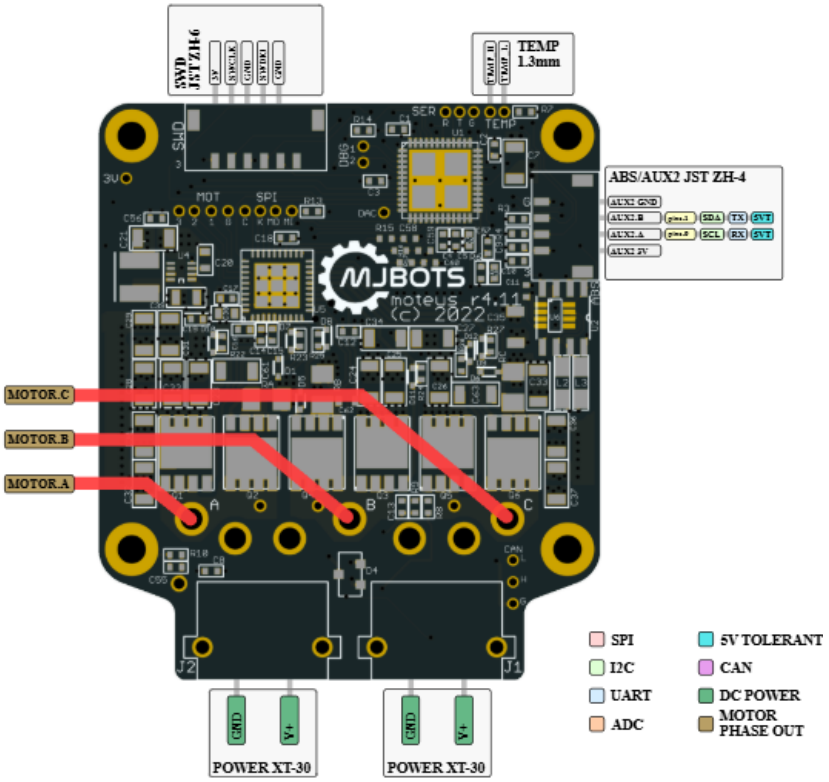


Figure 5.4: Connectors of the moteus r4.11 controller: power input, motor phases and CAN bus [30]

- **Power input.** The board is supplied through its **XT30** connector, from a battery pack at 24v . The power stage and the logic share this single input, so no auxiliary supply is required.
- **Motor phases.** The three phases of the mj5208 are wired to the `MOTOR.A`, `MOTOR.B` and `MOTOR.C` pads (Figure 5.4). Their order sets the direction of rotation, which is verified during the first rotation.
- **Position sensing.** A magnet fixed on the rear end of the motor shaft faces the magnetic sensor of the board, so the angular measurement is made inside the controller and no encoder cable is needed.
- **CAN-FD bus.** Two identical connectors allow the controllers to be daisy-chained on the same differential pair, terminated at both ends.

These connectors are assembled into the chain of Figure 5.5. The host computer runs `tview` and `moteus_tool` and reaches the controller through the mjcanfd-usb-1x adapter, which converts USB into CAN-FD. The controller drives the three phases of the mj5208

and receives its power on a separate line, so that a complete unit needs only two cables to operate: one for the bus and one for the power. This is what allows a winch to be added to the robot without modifying the units already installed.

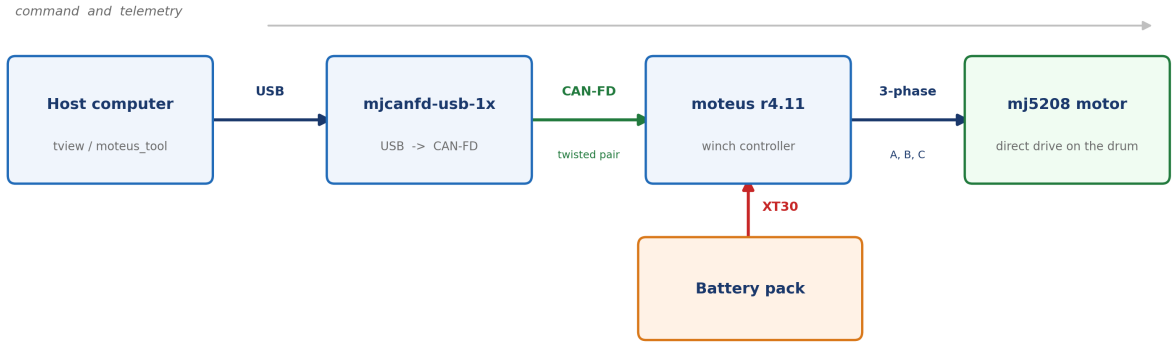


Figure 5.5: Wiring of the motor and interface with the host computer

The link established between the motor and the host computer is illustrated in Figure 5.6, together with the `tview` window through which the controller is commanded and its telemetry read.

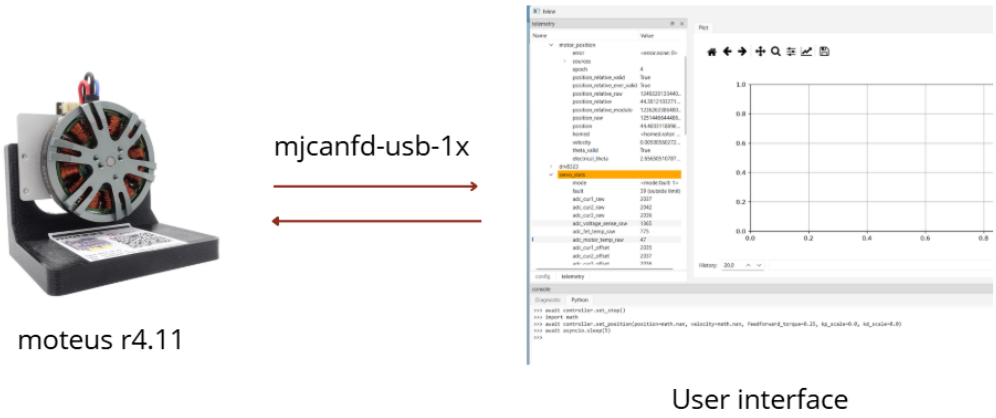


Figure 5.6: Interface between the motor and the host computer, reached through the mjcanfd-usb-1x adapter

5.2.3 Diagnostic and Configuration

The moteus software provides a graphical toolset built on top of its Python API, which gives access to the controller without writing any code: real-time visualisation of the telemetry, management of the configuration, and debugging [31]. This toolset is used here to check and configure the unit once it is wired. It is launched from the host computer with:

```
python3 -m moteus_gui.tview --devices=1
```

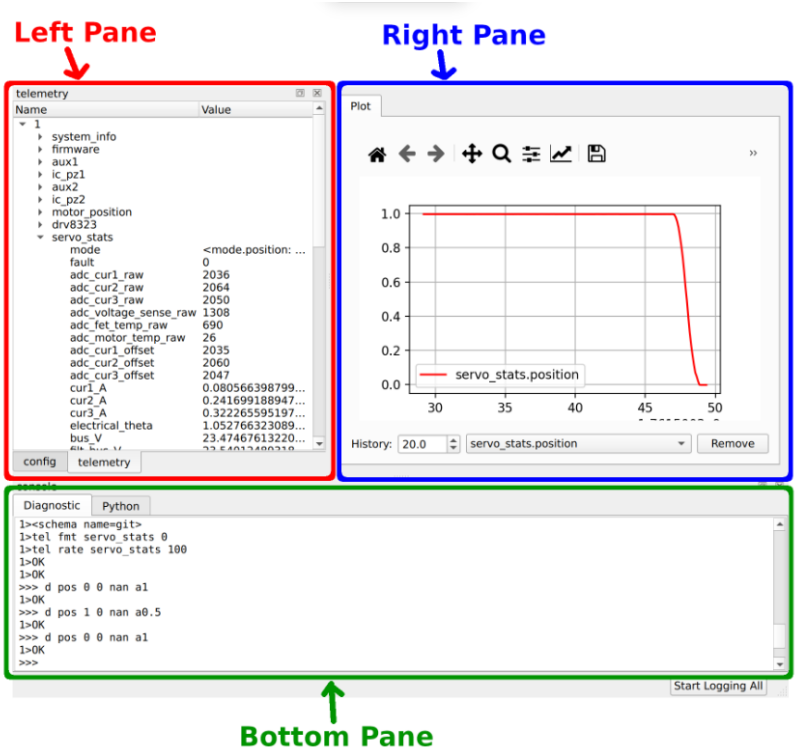


Figure 5.7: Graphical user interface `tview`

The interface gives access to three panes [32]:

- the **left pane**, a telemetry tree listing every internal variable of the controller;
- the **right pane**, which plots any telemetry item in real time;
- the **bottom pane**, the diagnostic console, from which the `conf set`, `conf write` and `d pos` commands are issued.

Diagnostic

Three points were verified before any rotation:

- the controller **answers on the bus** at the expected identifier;
- the **bus voltage** reported by the controller matches the one delivered by the power supply;
- the **fault register is at zero**.

Configuration

The parameters are written from the diagnostic console and stored in the non-volatile memory of the controller [33]:

`conf set <parameter> <value>` then `conf write`

- **Bus identifier** — identifies the controller on the CAN-FD bus; one identifier per winch.
- **Transmission ratio** — set to 1: the drum is mounted directly on the motor shaft, so one motor revolution is one drum revolution.
- **Current limit** — bounds the phase current, and therefore the torque, independently of the host.
- **Velocity limit** — bounds the drum speed, and therefore the cable speed.
- **Travel limits** — bound the drum rotation to the winding capacity of the groove, so the cable cannot be paid out beyond the grooved length.

The values retained for this winch are given in Table 5.2.

Table 5.2: Configuration parameters of the moteus controller

Parameter	**Value**
`id.id`	1
`motor_position.rotor_to_output_ratio`	1
`servo.max_current_A`	default
`servo.max_velocity`	0,9 rev/s
`servopos.position_min`	−22 rev
`servopos.position_max`	+22 rev

5.2.4 Motor Control

Before any control mode can be used, the firmware must know the relation between the magnetic poles of the rotor and the zero of the encoder. This relation is identified once with `moteus_tool --target 1 --calibrate`, which rotates the motor in open loop and stores the winding resistance, the phase inductance, the number of pole pairs and the encoder-to-pole offset in the persistent configuration of the controller [34]. The motor can then be commanded.

Control from tview

The motor was first rotated manually from the `tview` console, before writing any script. The command takes three arguments — target position, velocity and maximum torque:

```
d pos <position> <velocity> <max_torque>
```

A position set-point of 22 rev was commanded from the console, the drum being initially at 10 rev. The position reported by the controller was recorded in the plotting pane of `tview` (Figure 5.8).

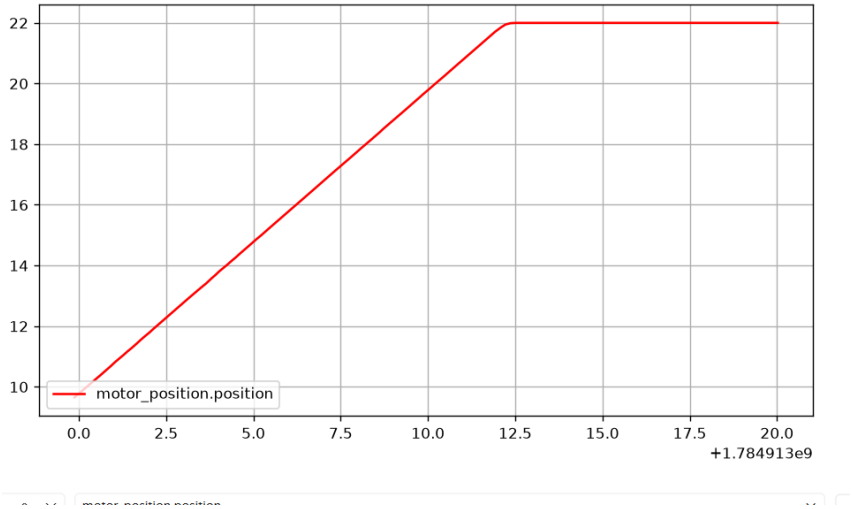


Figure 5.8: Position reported by the controller for a set-point of 22 rev, recorded in the plotting pane of `tview`

Control from the host in Python

The console command and the Python call write the same registers of the controller; the script only repeats it in a loop at a fixed rate. The command is issued through the `moteus` library:

```
set_position(position=p*, velocity=ω, maximum_torque=τ)
```

- **Position set-point:** the drum position in revolutions. Setting this field to `NaN` instead selects the velocity mode of Table 5.1: no position is imposed and the drum turns as long as the command is refreshed.
- **Velocity limit:** 5 rev/s.
- **Acceleration limit:** 2 rev/s^2, which bounds the current drawn at each change of set-point and limits the energy sent back to the power supply when the drum decelerates.

- **Torque limit:** 0.5 N·m, which bounds the cable tension the unit can apply.
- **Refresh rate:** the command is refreshed at 50 Hz. Every command frame is answered by a status frame carrying the position, the velocity, the current and the fault code, so the state of the motor is read back within the same exchange.

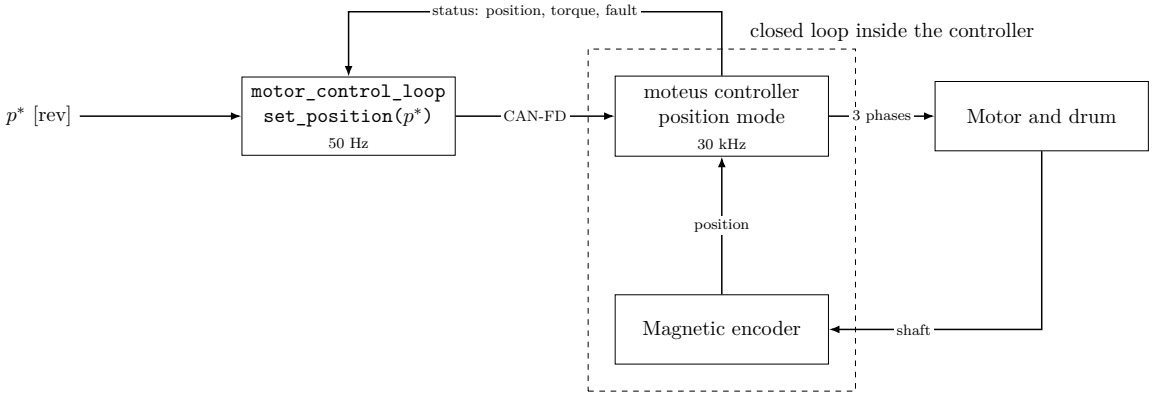


Figure 5.9: Position control of the motor from the host script

Table 5.3: Limits sent with each position command

Argument	**Value**
`velocity_limit`	5 rev/s
`accel_limit`	2 rev/s^2
`maximum_torque`	0.5 N·m
`watchdog_timeout`	0.5 s

5.3 Tension Acquisition Chain

The second chain brings the cable tension back to the host: the load cell measures the force on the guiding pulley, the HX711 amplifies and digitises it, and the Arduino board transmits the result over the serial link. It is wired, then calibrated, so that its gain and its offset are known before the tension is used as a feedback signal.

5.3.1 Load Cell and HX711 Wiring

Figure 5.10 shows the electrical wiring of the acquisition chain, connecting the load cell to the HX711 amplifier and the Arduino board. The corresponding connections are detailed in Table 5.4.

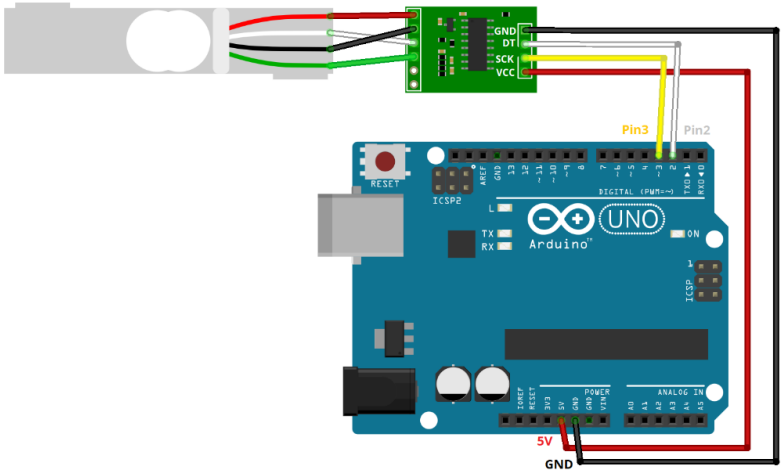


Figure 5.10: Wiring of the tension acquisition chain: load cell, HX711 amplifier and Arduino board

Table 5.4: Load cell, HX711 and Arduino wiring connections

Load cell → HX711		**HX711 → Arduino**	
Load cell	**HX711**	**HX711**	**Arduino**
Red (E+)	E+	VCC	**5V**
Black (E–)	E–	GND	**GND**
Green (A+)	A+	DT	Pin 2
White (A–)	A–	SCK	Pin 3

5.3.2 Calibration Principle

The measurement chain does not deliver a force. The bridge of the load cell produces a differential voltage of a few millivolts, which the HX711 amplifies and converts into a raw integer N. Calibration identifies the two constants that link this integer to the cable tension:

$$m = \frac{N - N_0}{k} \tag{5.1}$$

where:

- N : raw integer returned by the HX711 [counts];
- N_0 : offset, the raw reading with no additional load [counts];
- k : scale factor of the chain [counts/g];
- m : mass equivalent to the applied load [g].

The mass is then converted into the tension used by the control loop:

$$T = \frac{m}{1000}\, g, \qquad g = 9.81 \text{ m/s}^2 \tag{5.2}$$

The load cell was calibrated **in place**: mounted on its bracket at 45°, with the pulley support fitted, that is in the exact configuration in which it operates.

The two constants were identified with the `HX711` Arduino library, in three steps:

1. **Preparation.** The load cell is bolted on its 45° bracket and the pulley support is fitted, with no cable tension applied. The assembly is left at rest until the reading is stable.
2. **Offset.** The raw output is averaged in this unloaded state. The average is the offset N_0: it is the tare of the whole assembly — bracket, pulley and its bearing — so that the calibration reflects only the force added afterwards.
3. **Scale factor.** A reference mass m_{ref} is applied and the raw output averaged again, giving a reading N. Inverting equation (5.1) gives

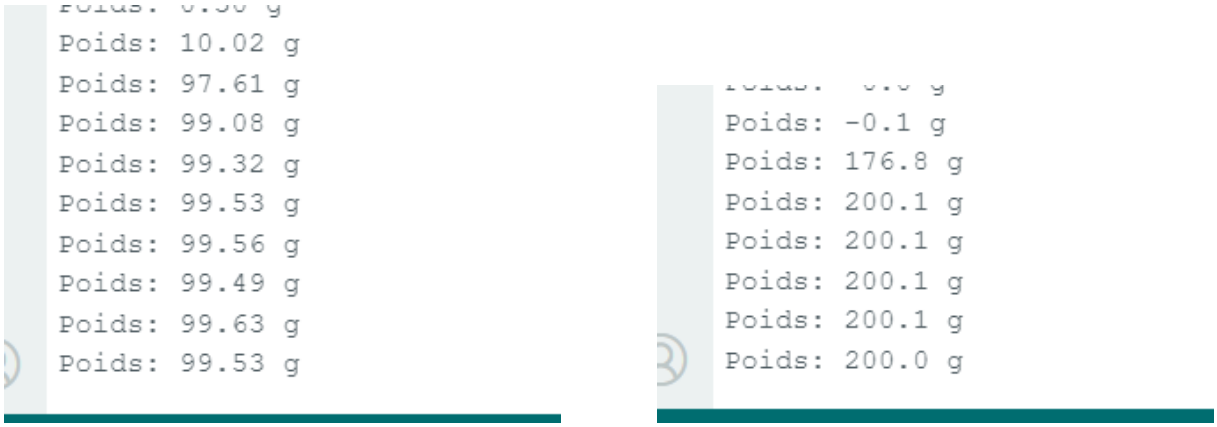
$$k = \frac{N - N_0}{m_{ref}} \tag{5.3}$$

Table 5.5: Identified constants and settings of the measurement chain

Quantity	**Symbol**	**Value**
Reference mass used	m_{ref}	100-200 g
Offset (tare of the assembly)	N_0	−370 825 counts
Scale factor	k	295.0 counts/g
Resolution of one count	—	3.3×10^{-5} N
Conversion rate of the HX711	—	10 samples/s
Serial baud rate	—	115 200 Bd
Serial transmission period	—	5 ms

5.3.3 Calibration Results

To validate the calibration, two reference weights of 100 g and 200 g were applied to the load cell and the calibrated readings were recorded (Figure 5.11). The comparison with the reference weights is given in Table 5.6.



(a) Reference weight of 100 g (b) Reference weight of 200 g

Figure 5.11: Calibrated readings recorded with the two reference weights

Table 5.6: Comparison between the reference weights and the measured values

Reference weight	Measured value	Absolute error	Relative error
100 g	99.5 g	0.5 g	0.50%
200 g	200.1 g	0.1 g	0.05%

5.4 Overview of the Experimental Setup

The two chains are gathered on the same bench, shown in Figure 5.12. The moteus controller is powered directly from the main power supply and communicates with the host computer via the CAN-FD to USB adapter (fdcanusb). In parallel, the load cell is connected to the HX711 amplifier, which interfaces with the Arduino board for real-time tension acquisition, itself connected to the host computer over a serial USB connection. This wiring configuration ensures that both the motor commands and the sensor readings are handled within the same control loop.

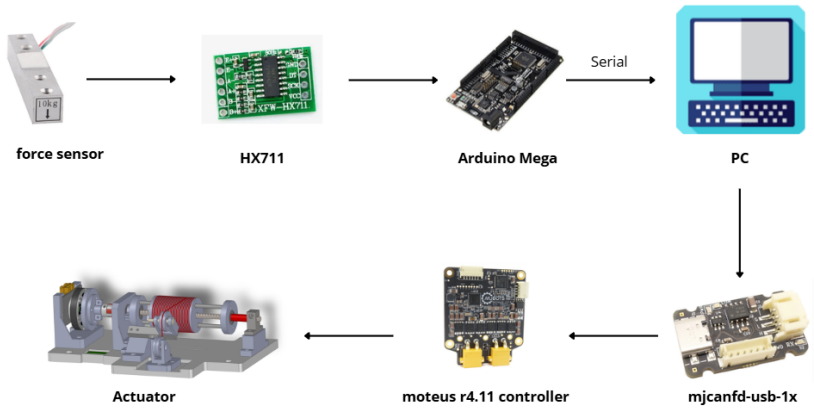


Figure 5.12: General view of the experimental setup

Conclusion

The winch designed in the previous chapter now exists as an assembled prototype, whose printed parts and standard components form a single self-contained unit. Its actuation chain reaches a commanded drum position with a controlled speed profile, the limits being enforced by the controller itself, while its acquisition chain returns a cable tension in grams validated against reference weights. The two chains being characterised independently, the next chapter closes the loop between them and presents the control strategies of the winch.

Chapter 6

Control System

Introduction

The previous chapter delivered a functional winch whose two chains are characterised: the actuation chain drives the drum, and the acquisition chain returns the cable tension. This chapter closes the loop between them. The tension control is presented first, with its structure, its controller, its safety layer and its experimental validation, then the position control, and finally the hybrid strategy that a cable-driven parallel robot requires.

6.1 Tension Control

6.1.1 Control Strategy

Only one winch unit was built, so the cable could not be loaded by a moving platform. The tension is applied instead by a **known mass suspended at the free end of the cable**, after the swivel pulley (Figure 6.1). This arrangement gives a load that is exactly known, perfectly repeatable, and produced independently of the controller — the three properties required to judge a control loop.

At rest, the suspended mass imposes the tension in the cable:

$$T = m\,g \tag{6.1}$$

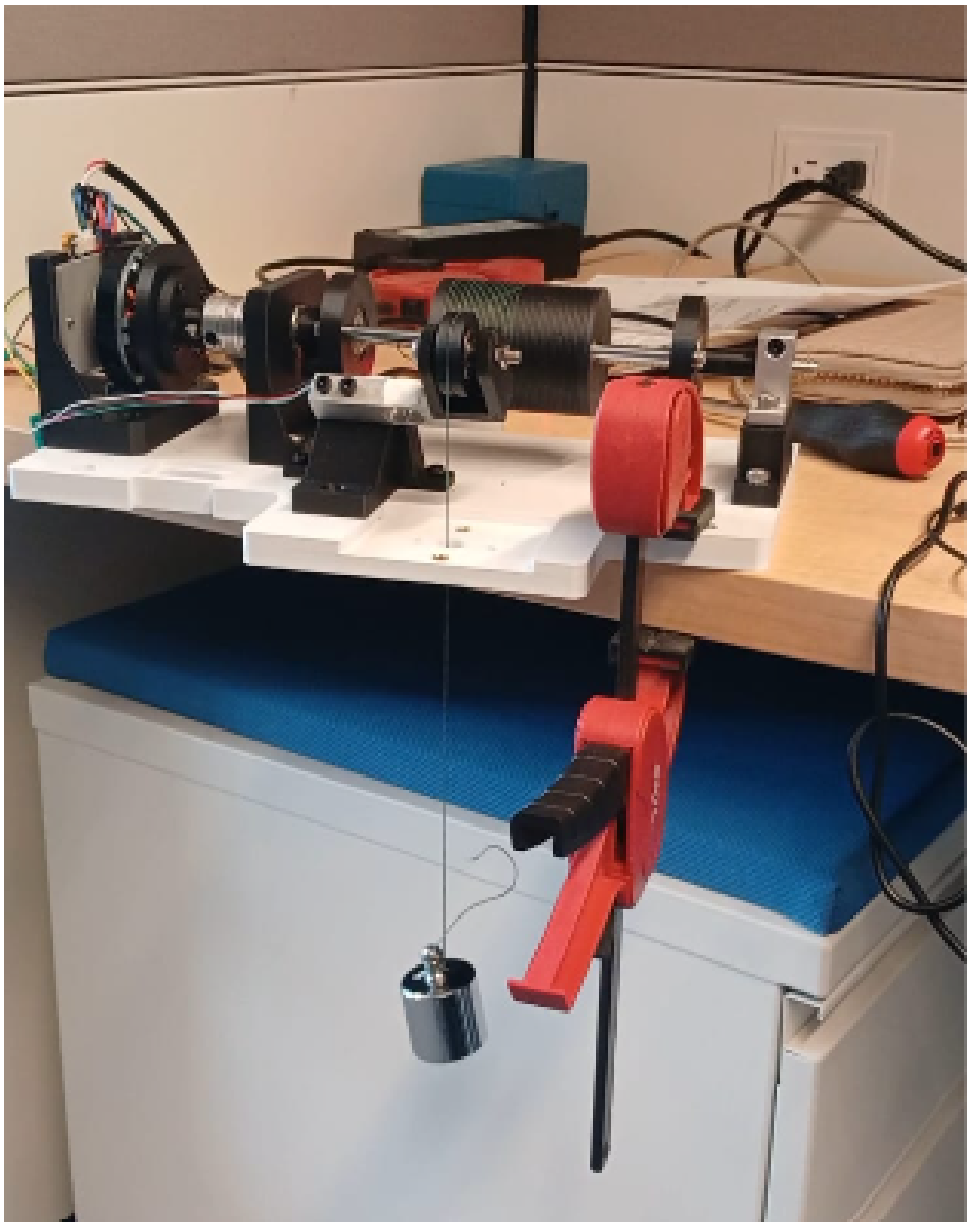


Figure 6.1: Test bench used to regulate the cable tension. The mass suspended at the free end of the cable, after the swivel pulley, imposes a known tension $T = m\,g$

The tension control strategy was implemented using a **PI controller**, regulating the motor's output velocity based on the real-time tension feedback provided by the load cell. The moteus controller was operated in **velocity mode** — using the `set_position` command with the position argument set to `NaN` — allowing direct control of the motor's rotational velocity to track the desired cable tension indirectly.

The principle is the following: at each cycle, the tension measured by the load cell is compared with the desired set-point. If the tension is too low, the drum winds the cable in and the tension increases; if the tension is too high, the drum pays the cable out and the tension decreases. The output of the controller is therefore not a tension but a *winding velocity*.

A direct torque command was also considered, since in direct drive the torque is proportional to the tension. This solution was not retained: it is applied in open loop with respect to the tension, so any friction in the bearings or in the screw–nut mechanism appears directly as a tension error, and the load cell measurement is not used at all. The velocity-based solution exploits the feedback at every cycle and is therefore insensitive to these unmodelled effects.

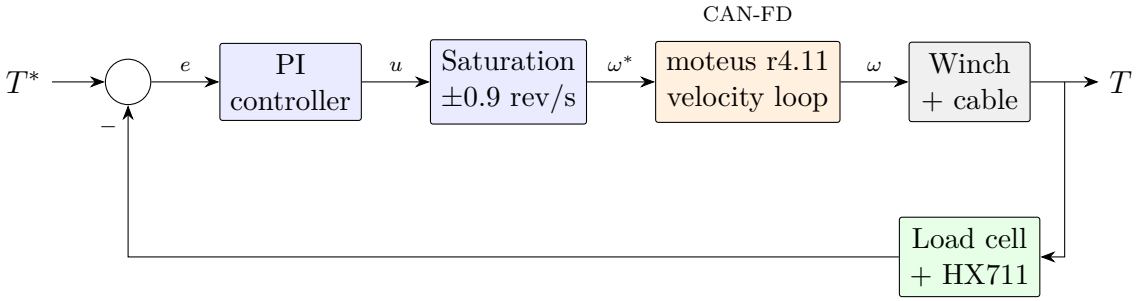


Figure 6.2: Block diagram of the tension control loop

As shown in Figure 6.2, the control system is organised as a **cascade of two loops**: the outer tension loop, executed on the host computer at 50 Hz, which computes the velocity command from the tension error, and the inner velocity and current loop, executed inside the moteus controller at 30 kHz.

6.1.2 PI Controller

The controller computes the velocity command from the tension error, defined as the difference between the setpoint T^* and the measurement T:

$$e(t) = T^*(t) - T(t) \tag{6.2}$$

$$\omega^*(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau \tag{6.3}$$

A derivative term was deliberately not used. The derivative acts on the variation of the error, so it directly amplifies the noise of the load cell measurement, and the delay introduced by the acquisition chain makes its contribution unreliable.

The gains have unusual units, since the controller converts a force into a velocity: K_p is expressed in (rev/s)/N and K_i in (rev/s)/(N · s).

The controller is discretised at $T_s = 20$ ms using a rectangular approximation of the integral:

$$I_k = I_{k-1} + e_k\, T_s \tag{6.4}$$

$$u_k = K_p\, e_k + K_i\, I_k \tag{6.5}$$

Three protections are added to the control. The integral term is clamped to $I_{max} = 3.0$ to prevent windup during saturation, the velocity command is limited to ± 0.9 rev/s, and a deadband of $\varepsilon = 0.05$ N is applied around the setpoint so that the motor stops once the target tension is reached instead of dithering on the measurement noise.

The value of the deadband is set from the measurement: the noise band of the acquisition chain characterised in Chapter 5 is of the order of 0.01 N, so a threshold of 0.05 N is safely above the noise floor while remaining small compared with the operating tensions of the winch.

6.1.3 Software Implementation

The control loop was implemented in **Python**, using an asynchronous (`asyncio`) architecture, running at a control rate of **50 Hz**. At each control cycle, the current tension measurement is compared to the desired setpoint, and the resulting error is used by the PI controller to compute the appropriate velocity command sent to the motor controller over CAN-FD.

Software architecture

The software is organised in three concurrent blocks running on a single `asyncio` event loop and exchanging data through a shared state object (Figure 6.3). This structure avoids the cost and the race conditions of true multithreading, while allowing the blocking operations (serial read, keyboard input) to be delegated to an executor so that they never stall the control loop.

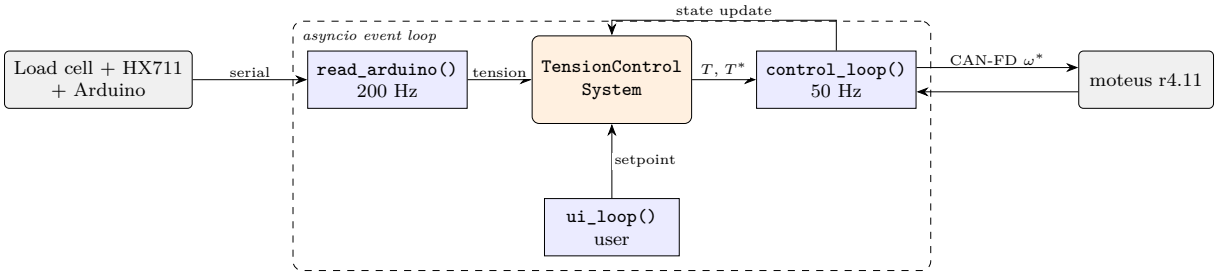


Figure 6.3: Block architecture of the control software

The role of each block is detailed in Table 6.1.

Table 6.1: Description of the three concurrent software blocks

Block	**Rate**	**Function**
`read_arduino()`	200 Hz	Polls the serial port, decodes the load cell reading, converts it into newtons.
`control_loop()`	50 Hz	Executes the PI computation, applies the safety guards, sends the velocity command over CAN-FD and logs the time, the measured tension and the set-point.
`ui_loop()`	On demand	Reads the operator commands from the keyboard (new setpoint) without blocking the control loop.

▶ The acquisition task runs faster than the sensor itself: the HX711 delivers about 10 conversions per second, so a new value reaches the controller roughly once every twenty cycles. This ratio, and not the control rate, is what bounds the bandwidth achievable by the outer loop.

Steps of one control cycle

Each control cycle executes the sequence of steps shown in Figure 6.4. The safety guards are deliberately placed *after* the controller computation and *before* the transmission of the command, so that no controller output can bypass them — including the zero command produced by the deadband branch.

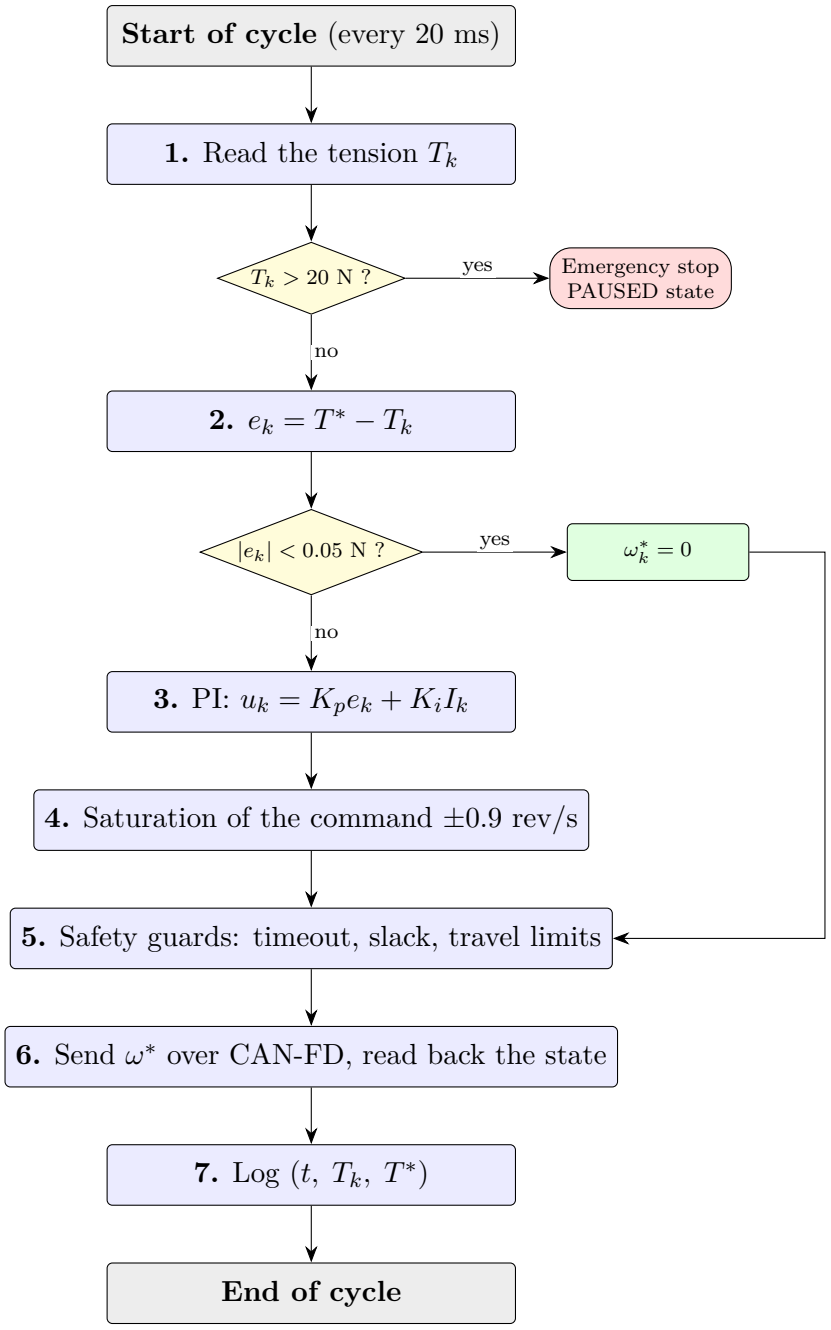


Figure 6.4: Steps executed at each control cycle

Controller parameters

The complete set of parameters used by the control loop is given in Table 6.2.

Table 6.2: Parameters of the tension controller

Parameter	**Symbol**	**Value**
Proportional gain	K_p	0.70
Integral gain	K_i	0.02
Integral clamp	I_{max}	3.0
Deadband	ε	0.05 N
Velocity saturation	ω_{max}	0.9 rev/s
Acceleration limit	$\dot{\omega}_{max}$	0.3 rev/s^2
Torque limit	τ_{max}	1.0 N·m
Control period	T_s	20 ms (50 Hz)

6.1.4 Safety Management

A supervision layer was implemented *around* the regulation loop. It is applied after the controller computation and before the command is transmitted, so that no controller output can bypass it. The implemented protections are summarised in Table 6.3.

Table 6.3: Safety mechanisms of the tension control system

Mechanism	**Threshold**	**Action**
Overload protection	$T > 20$ N	Emergency stop, PAUSED state
Slack detection	$T < 0$ N and $\omega^* < 0$	Unwinding inhibited
Travel limit	$\lvert x \rvert > 45$ mm	Motion in that direction inhibited
Correction timeout	> 50 s outside deadband	Emergency stop
Communication watchdog	0.5 s without command	Motor stopped by moteus
Torque ceiling	1 N·m	Enforced by the firmware
Velocity / accel. ceiling	0.9 rev/s, 0.3 rev/s^2	Enforced by the firmware
Fault monitoring	Fault register $\neq 0$	Reported to the operator
CAN-FD exception	Any transaction failure	Emergency stop

Slack detection. A tension falling to zero while the cable is being paid out indicates ground contact or a released cable. Unwinding is inhibited to prevent a loose coil from forming on the drum.

Correction timeout. If the error stays outside the deadband for more than 50 s, the setpoint is considered unreachable (blockage, broken cable or sensor failure) and the system stops.

Communication watchdog. Since Python on a general-purpose operating system does not guarantee a strictly periodic loop, a 0.5 s watchdog on the moteus controller stops the motor if no command is received in time.

6.1.5 Experimental Results

The reference is imposed by a calibrated mass suspended at the free end of the cable. For $m = 200$ g, the equilibrium of the load gives

$$T_{ref} = m\,g = 0.200 \times 9.81 = 1.962 \text{ N} \tag{6.6}$$

This value is written into the controller as the set-point, $T^* = 1.960$ N. Once the transient is over, the measured tension stays between 1.945 and 1.951 N (Figure 6.5), that is a mean error of **0.5** % of the set-point.

```
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.951N | TORQUE: cmd=0.0703Nm actual=0.0703Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.950N | TORQUE: cmd=0.0725Nm actual=0.0717Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.950N | TORQUE: cmd=0.0725Nm actual=0.0739Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.949N | TORQUE: cmd=0.0746Nm actual=0.0754Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.949N | TORQUE: cmd=0.0746Nm actual=0.0768Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.951N | TORQUE: cmd=0.0688Nm actual=0.0726Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.951N | TORQUE: cmd=0.0688Nm actual=0.0662Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.947N | TORQUE: cmd=0.0788Nm actual=0.0767Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.947N | TORQUE: cmd=0.0788Nm actual=0.0782Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.954N | TORQUE: cmd=0.0641Nm actual=0.0649Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.954N | TORQUE: cmd=0.0641Nm actual=0.0622Nm
mode=10 fault=0 stall=0 pos=-18.77turns(~-37.5mm) | TENSION: target=1.960N measured=1.945N | TORQUE: cmd=0.0817Nm actual=0.0811Nm
```

Figure 6.5: Tension regulated at the set-point $T^* = 1.960$ N for a suspended mass of 200 g

6.2 Hybrid Control

6.2.1 Control Strategy

Position control alone brings the cable to the required length but ignores the tension, so the cable may go slack or become overloaded. Tension control alone maintains the force but cannot follow a trajectory. The hybrid strategy combines both: the two controllers run simultaneously and their outputs are merged into a single velocity command through a **weighted blending** scheme.

The weights are not fixed and the position keeps priority. Far from the target, $w_p = 1$: the position controller has full authority and the winch acts as a positioning actuator. Inside the blending distance, the authority is transferred progressively to the tension controller, which takes over completely at the target ($w_t = 1$), the winch then acting as a constant-force actuator. The transfer is reversible and recomputed at every cycle: any growth of the position error immediately restores authority to the position controller. Being progressive rather than a hard switch, it introduces no discontinuity in the velocity command.

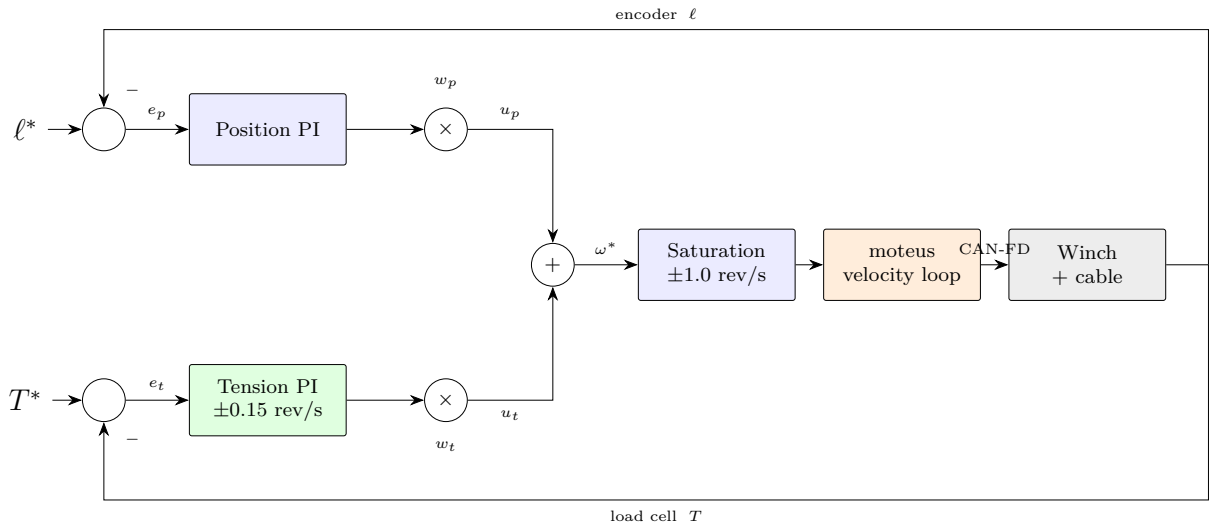


Figure 6.6: Block diagram of the hybrid control loop

6.2.2 Weighting Function

The position weight w_p is computed at every cycle from the remaining position error e_p, normalised by a blending distance $d_b = 50$ mm:

$$w_p = \min\left(1,\ \frac{|e_p|}{d_b}\right) \tag{6.7}$$

$$w_t = 1 - w_p \tag{6.8}$$

Beyond 50 mm from the target the position controller has full authority ($w_p = 1$), and inside this distance the authority is transferred linearly to the tension controller, which takes over completely when the target position is reached (Figure 6.7).

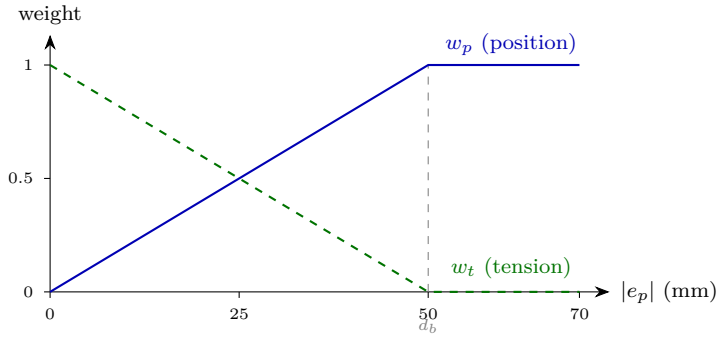


Figure 6.7: Evolution of the position and tension weights with the position error

6.2.3 Position and Tension Controllers

The winch position is expressed in motor revolutions. Since the grooved drum imposes a constant winding radius, the deployed cable length is proportional to the drum rotation: one revolution releases the circumference of the drum,

$$\ell = \pi D_{drum}\, n = 163.4\, n \quad [\text{mm}] \tag{6.9}$$

where n is the number of drum revolutions counted by the magnetic encoder of the controller. This relation is purely geometric: it ignores the elastic elongation of the cable under tension and its sag under its own weight, which make the real distance to the platform slightly larger than the length paid out.

The position error is therefore expressed directly in millimetres of cable,

$$e_p = \ell^* - \ell \tag{6.10}$$

so that the blending distance d_b, the position deadband and the travel limits are all in the same unit. The gains $K_{p,p}$ and $K_{i,p}$ are consequently expressed in (rev/s)/mm and (rev/s)/(mm · s).

The position controller is a PI controller with clamped integral:

$$u_p = K_{p,p}\, e_p + K_{i,p} \int e_p\, dt \tag{6.11}$$

The tension controller keeps the same PI structure as in Section 6.1, but its output is limited to a much tighter velocity range of ±0.15 rev/s:

$$u_t = \text{sat}\Big(K_{p,t}\, e_t + K_{i,t} \int e_t\, dt,\ \pm 0.15 \Big) \tag{6.12}$$

This tight limit is essential: without it the tension correction could saturate at the full velocity limit and produce a bang-bang oscillation of the cable during the blending phase.

The two corrections are finally merged and saturated to produce the velocity command sent to the motor:

$$\omega^* = \text{sat}\Big(w_p\, u_p + w_t\, u_t,\ \pm 1.0 \Big) \tag{6.13}$$

Each controller keeps its own deadband — 0.5 mm on the position and 0.08 N on the tension — inside which its correction and its integrator are reset to zero.

6.2.4 Implementation

The hybrid controller reuses the asynchronous architecture of Section 6.1: the same three concurrent blocks, the same 50 Hz rate and the same velocity-mode command over CAN-FD. Two changes are introduced: the integral term is computed with the **measured** elapsed time rather than the nominal period, since the CAN-FD transaction is blocking and its duration varies, and the tension is filtered by a **median of the last five readings**, which rejects the isolated outliers of the load cell without the lag of a longer averaging window.

Table 6.4: Parameters of the hybrid controller

Parameter	**Symbol**	**Value**
Blending distance	d_b	50 mm
Position proportional gain	$K_{p,p}$	1.0
Position integral gain	$K_{i,p}$	0.5
Position integral clamp	—	0.3
Position deadband	—	0.5 mm
Tension proportional gain	$K_{p,t}$	0.02
Tension integral gain	$K_{i,t}$	0.01
Tension integral clamp	—	3.0
Tension deadband	—	0.08 N
Tension velocity cap	—	0.15 rev/s
Global velocity saturation	ω_{max}	1.0 rev/s
Acceleration limit	$\dot{\omega}_{max}$	0.3 rev/s^2
Torque limit	τ_{max}	1.0 N·m
Control period	T_s	20 ms (50 Hz)

6.2.5 Safety Management

The hybrid controller keeps the protections of the tension loop and adds those listed in Table 6.5.

Table 6.5: Additional safety mechanisms of the hybrid controller

Mechanism	**Threshold**	**Action**
Overload protection	$T > 5$ N	Emergency stop
Slack protection	$T < 0.15$ N	Unwinding inhibited
Travel limits	0 – 2000 mm	Motion inhibited in that direction
Stall detection	$\tau > 0.3$ N·m and $\lvert\omega\rvert < 0.05$	Emergency stop
Correction timeout	> 80 s outside deadband	Emergency stop
Loop error counter	10 consecutive failures	Emergency stop

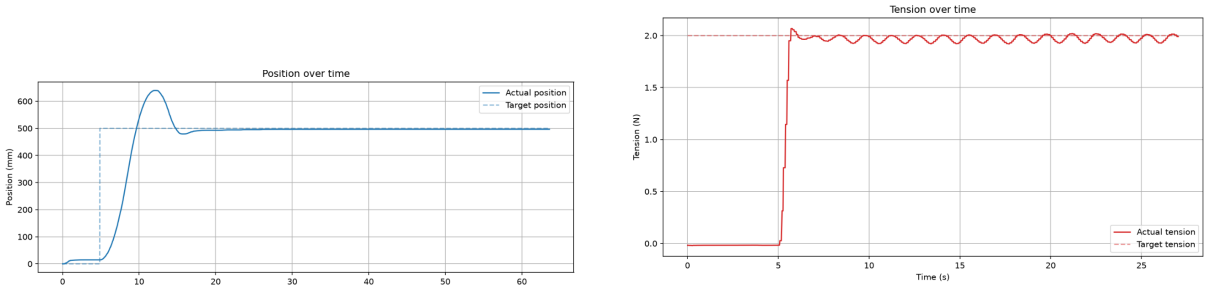
- **Slack protection.** Unwinding is refused below 0.15 N, because a cable with no tension leaves its grooves and tangles on the drum.
- **Stall detection.** A high torque together with an almost zero velocity means the drum is blocked, a situation the drive does not report as a fault and which would make the motor overheat.

6.2.6 Experimental Results

The test consists in commanding a position setpoint while a tension setpoint is maintained, and recording both variables simultaneously at 50 Hz.

```
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=1.99N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=1.99N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=2.02N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=2.02N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=2.02N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=0.00N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=2.02N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.5mm error=0.5mm (w=0.02) | TENSION: target=2.00N measured=2.04N (w=0.98)
mode=10 fault=0 stall=0 | POS: target=400.0mm actual=399.6mm error=0.4mm (w=0.02) | TENSION: target=2.00N measured=2.04N (w=0.98)
```

Figure 6.8: Steady state of the hybrid controller: the position is held 0.5 mm from its target and the tension 0.04 N from its setpoint, with the authority transferred to the tension loop



(a) Position (b) Tension

Figure 6.9: Position and tension during a hybrid control test

Figure 6.9 shows that both objectives are reached.

- **Position.** The winch follows the setpoint with a single overshoot of about , that is , then settles after roughly and stays inside the 0.5 mm deadband until the end of the test.
- **Tension.** The cable reaches its setpoint of 2.00 N and is held there for the rest of the run, with a residual ripple of . The steady state of Figure 6.8 gives 1.99 to 2.04 N, that is a maximum error of 0.04 N, well inside the 0.08 N deadband.

▶ The blending law behaves as designed: far from the target the position controller drives the motion, and once the target is reached the authority passes to the tension controller, which holds the force ($w_p = 0.02$, $w_t = 0.98$ in Figure 6.8). The winch therefore reaches a commanded cable length *and* maintains a commanded tension at the same time, which is the objective of the hybrid strategy.

Conclusion

Three control strategies were implemented on the winch and evaluated on the experimental bench: a tension regulation built on a PI controller acting on the winding velocity, a position regulation exploiting the constant winding radius of the grooved drum, and a hybrid strategy blending the two according to the remaining position error. The tests confirm that the position objective is reached with no steady-state error, while the tension regulation in hybrid mode remains limited by the output rate of the acquisition chain. The unit is therefore operational as a self-contained actuator, and the improvements identified here define the work that remains before its integration into a complete cable-driven parallel robot.

General Conclusion

The objective of this work was to design, build and test a **self-contained winch** for a cable-driven parallel robot: a module gathering, inside a single envelope, the actuation, the storage and the guidance of the cable, and the measurement of its tension.

Results obtained

- **Mechanical design.** The unit stores 5 m of cable in a single layer on a grooved drum of 52 mm diameter. The drum **rotates and translates** on a grounded lead screw, so the cable always leaves the unit at the same point with a single actuator.
- **Integrated tension measurement.** A bending-beam load cell carries the measuring pulley inside the unit. Calibrated in place, on its support and with the pulley mounted, it returns the cable tension.
- **Self-contained electronics.** The controller is mounted on the rear face of the motor and the angular measurement is made on the board, so only two cables leave the winch: the power supply and the CAN-FD pair. Adding a winch to the robot means adding a node on the existing bus.
- **Validated actuation.** The prototype reaches a commanded drum position with a controlled speed profile, the travel, velocity and torque limits being enforced by the firmware independently of the host.
- **Control strategies.** Two controllers were implemented and evaluated on the bench: a tension regulation, and a hybrid strategy that blends a length command and a tension command according to the remaining position error.

Perspectives

Three directions are proposed to continue this work:

- **Increase the measurement rate**, by using the faster output mode of the converter or a different amplifier, which would remove the main obstacle to a wider bandwidth

of the tension regulation;

- **Duplicate the unit** and build the complete robot on the pegboard frame, in order to validate experimentally the coordination that the simulation predicts;
- **Distribute the tension** between the eight cables in real time, so that the redundancy of the robot is used to keep every cable taut over the whole workspace.

The winch developed in this project is therefore operational as an individual module, and the elements needed to replicate it — drawings, source code and data — are given in the appendices.

Bibliography

[1] mjbots Robotic Systems, "mj5208 brushless motor." https://mjbots.com/products/mj5208, 2024. Accessed: 2025-08-06.

[2] mjbots Robotic Systems, "moteus r4.11 controller." https://mjbots.com/products/moteus-r4-11, 2024. Accessed: 2025-08-06.

[3] mjbots Robotic Systems, "mjbots CAN-FD USB adapter (mjcanfd-usb-1x)." https://mjbots.com/products/mjcanfd-usb-1x, 2024. Accessed: 2025-08-06.

[4] Wishiot, "Digital load cell weight sensor 1 kg with HX711 amplifier." https://www.amazon.ca/dp/B07MTYS3XJ, 2024. Accessed: 2025-08-06.

[5] Amazon.ca, "Tr8x2 lead screw with T8 brass nut, acme thread, 2 mm pitch, 1 start, 2 mm lead." https://www.amazon.ca/dp/B0B8RKLC6F, 2024. Accessed: 2026-08-26.

[6] Amazon.ca, "uxcell Linear Motion Rod Shaft Guide, 8 x 150 mm, Case Hardened Chrome Plated Steel." https://www.amazon.ca/dp/B0GCDN4C9P, 2024. Accessed: 2026-08-26.

[7] A. Pott, *Cable-Driven Parallel Robots: Theory and Application*, vol. 120 of *Springer Tracts in Advanced Robotics*. Cham: Springer, 2018.

[8] Tacuna Systems, "An overview of load cells." https://tacunasystems.com/knowledge-base/an-overview-of-load-cells/, 2026. Accessed: 2026-08-26.

[9] Avia Semiconductor, "HX711 — 24-Bit Analog-to-Digital Converter (ADC) for Weigh Scales." https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf, 2024. Accessed: 2026-08-26.

[10] mjbots, "moteus reference — control modes." https://github.com/mjbots/moteus/blob/main/docs/reference.md, 2024. Accessed: 2026-08-26.

[11] Mitacs, "Globalink research internship." https://www.mitacs.ca. Accessed: 2025-11-15.

[12] Ontario Tech University, "Faculty of engineering and applied science." `https://ontariotechu.ca`. Accessed: 2025-11-15.

[13] Z. Zhang, Z. Shao, Z. You, X. Tang, B. Zi, G. Yang, C. Gosselin, and S. Caro, "State-of-the-art on theories and applications of cable-driven parallel robots," *Frontiers of Mechanical Engineering*, vol. 17, no. 3, p. 37, 2022.

[14] R. Wang, J. Li, and Y. Li, "A review on design, modeling and control technology of cable-driven parallel robots," *Robotics*, vol. 14, no. 9, p. 116, 2025.

[15] T. P. Tho and N. T. Thinh, "An overview of cable-driven parallel robots: Workspace, tension distribution, and cable sagging," *Mathematical Problems in Engineering*, vol. 2022, p. 2199748, 2022.

[16] S. Cao, Z. Luo, and C. Quan, "Real-time tension distribution design for cable-driven parallel robot," *Applied Sciences*, vol. 13, no. 1, p. 10, 2023.

[17] E. Idà and V. Mattioni, "Cable-driven parallel robot actuators: State of the art and novel servo-winch concept," *Actuators*, vol. 11, no. 10, p. 290, 2022.

[18] J.-H. Meusener, K. Kollek, J.-C. Krabbe, and A. Kummert, "Developing a winch for a low-cost cable-driven parallel robot," in *2024 IEEE International Conference on Smart Mechatronics (ICSMech)*, pp. 129–135, IEEE, 2024.

[19] V. Mattioni, E. Idà, M. Gouttefarde, and M. Carricato, "A practical approach for the hybrid joint-space control of overconstrained cable-driven parallel robots," in *Cable-Driven Parallel Robots*, vol. 132 of *Mechanisms and Machine Science*, pp. 149–160, Cham: Springer, 2023.

[20] X. Jin, H. Zhang, L. Wang, and Q. Li, "Review on control strategies for cable-driven parallel robots with model uncertainties," *Chinese Journal of Mechanical Engineering*, vol. 37, no. 1, p. 156, 2024.

[21] W. Kraus, P. Miermeister, V. Schmidt, and A. Pott, "Hybrid position/force control of a cable-driven parallel robot with experimental evaluation," in *New Trends in Mechanism and Machine Science*, vol. 24 of *Mechanisms and Machine Science*, pp. 553–561, Cham: Springer, 2015.

[22] J. Piao, E.-S. Kim, H. Choi, C.-B. Moon, E. Choi, J.-O. Park, and C.-S. Kim, "Indirect force control of a cable-driven parallel robot: Tension estimation using artificial neural network trained by force sensor measurements," *Sensors*, vol. 19, no. 11, p. 2520, 2019.

[23] International Organization for Standardization, "ISO 11898-1:2015 — road vehicles — controller area network (CAN) — part 1: Data link layer and physical signalling." https://www.iso.org/standard/63648.html, 2015.

[24] mjbots, "Raw CAN-FD — moteus documentation." https://mjbots.github.io/moteus/integration/can-fd/, 2024. Accessed: 2025-08-06.

[25] Amazon.ca, "MECCANIXITY Sleeve Bearing, 6 mm Bore x 8 mm OD x 6 mm Length, Self-Lubricating Cast Brass Bushing." https://www.amazon.ca/dp/B0FFSDFDJW, 2024. Accessed: 2026-08-26.

[26] Amazon.ca, "SK8 Linear Motion Rail Clamping Guide Support, Aluminium Shaft Support Bracket for 8 mm Diameter Shafts." https://www.amazon.ca/dp/B0FH54PP5M, 2024. Accessed: 2026-08-26.

[27] Amazon.com, "Saiper Flexible Coupling 6 mm to 8 mm, Aluminum Alloy Joint Connector for Stepper Motors and 3D Printers." https://www.amazon.com/dp/B07S8XJT4D, 2024. Accessed: 2026-08-26.

[28] Unirope Ltd., "Sheave- and drum dimensions." https://www.unirope.com/sheave-drum-dimensions/, 2023. Accessed: 2025-08-06.

[29] mjbots Robotic Systems, "moteus controller — theory of operation." https://github.com/mjbots/moteus/blob/main/docs/theory_of_operation.md. Accessed: September 7, 2026.

[30] mjbots, "Pinouts — moteus documentation." https://mjbots.github.io/moteus/reference/pinouts/, 2024. Accessed: 2026-08-26.

[31] mjbots, "moteus-gui: the tview graphical tool." https://pypi.org/project/moteus-gui/, 2025. Accessed: 2025-08-06.

[32] mjbots, "Getting started — moteus." https://deepwiki.com/mjbots/moteus/2-getting-started, 2024. Accessed: 2025-08-06.

[33] mjbots, "Configuration — moteus documentation." https://mjbots.github.io/moteus/guides/configuration/, 2024. Accessed: 2025-08-06.

[34] mjbots, "Calibration — moteus documentation." https://mjbots.github.io/moteus/guides/calibration/, 2024. Accessed: 2025-08-06.

APPENDICES

Appendix A — Source Code

The programs written for this work are reproduced below.

A.1 Calibration of the Load Cell

This script identifies the offset N_0 and the scale factor k defined in Section 5.3.2.

```
#include "HX711.h"

const int LOADCELL_DOUT_PIN = 2;
const int LOADCELL_SCK_PIN = 3;

HX711 scale;

const long CALIBRATION_OFFSET = -370825;
const float CALIBRATION_FACTOR = 295.0;

void setup() {
  Serial.begin(115200);
  scale.begin(LOADCELL_DOUT_PIN, LOADCELL_SCK_PIN);
  scale.set_offset(CALIBRATION_OFFSET);
  scale.set_scale(CALIBRATION_FACTOR);

  unsigned long start_wait = millis();
  while (!scale.is_ready()) {
    if (millis() - start_wait > 5000) {

      start_wait = millis();
    }
    delay(100);
  }

}

void loop() {
  if (scale.is_ready()) {

    float grams = scale.get_units(1);
    Serial.println(grams);
  }
  delay(5);
}
```

Listing 6.1: Calibration of the load cell (Arduino)

A.2 Hybrid Control

This script implements the hybrid control strategy of Section 6.2. Only the control class is reproduced; the serial reading, the user interface and the logging routines are omitted.

```
import asyncio
import moteus
import serial
import math
import traceback

ARDUINO_PORT = "COM4"
ARDUINO_BAUD = 115200
GRAVITY = 9.81

DRUM_DIAMETER_MM = 52.0
DRUM_RADIUS_M = (DRUM_DIAMETER_MM / 1000.0) / 2.0
CIRCUMFERENCE_MM = 2 * math.pi * DRUM_RADIUS_M * 1000

MAX_TRAVEL_MM = 2000
POSITION_MIN_TURNS = 0.0
POSITION_MAX_TURNS = MAX_TRAVEL_MM / CIRCUMFERENCE_MM
BLEND_DISTANCE_MM = 50

POSITION_DEADBAND_MM = 0.5
POSITION_DEADBAND_TURNS = POSITION_DEADBAND_MM / CIRCUMFERENCE_MM

KP_POSITION = 1
KI_POSITION = 0.5
POSITION_INTEGRAL_LIMIT = 0.3

TENSION_DEADBAND_N = 0.08
KP_TENSION = 0.02
KI_TENSION = 0.01
KD_TENSION = 0
TENSION_INTEGRAL_LIMIT = 3.0
TENSION_MAX_VELOCITY = 0.15

SAFE_VELOCITY = 1.0
MAX_TORQUE_LIMIT = 1
ACCEL_LIMIT = 0.3

MAX_SAFE_TENSION_N = 5.0
MIN_SAFE_TENSION_N = 0.15
CORRECTION_TIMEOUT_S = 80

STALL_TORQUE_THRESHOLD = 0.3
STALL_VELOCITY_THRESHOLD = 0.05
STALL_COUNT_LIMIT = 25

running = True

log_times = []
log_positions = []
log_target_positions = []
log_tensions = []
log_target_tensions = []

```

```
class WeightedBlendControl:
    def __init__(self, controller):
        self.controller = controller
        self.target_position_turns = 0.0
        self.target_tension_n = 2.0

        self.velocity_cmd = 0.0
        self.actual_velocity = 0.0
        self.actual_torque_nm = 0.0
        self.position = 0.0
        self.mode = 0
        self.fault = 0
        self.stall_counter = 0

        self.position_integral = 0.0
        self.tension_integral = 0.0
        self.measured_tension_n = 0.0
        self.correction_start_time = None

        self.position_weight = 1.0
        self.tension_weight = 0.0

        self._last_update_time = None

        self._prev_tension_error = 0.0

        self._recent_tension_readings = []

        self._smoothed_tension_n = None

    def mm_to_turns(self, length_mm):
        return length_mm / CIRCUMFERENCE_MM

    def update_measured_tension(self, grams):
        new_tension_n = (grams / 1000.0) * GRAVITY

        self._recent_tension_readings.append(new_tension_n)
        if len(self._recent_tension_readings) > 5:
            self._recent_tension_readings.pop(0)
        if len(self._recent_tension_readings) < 5:
            self.measured_tension_n = new_tension_n
        else:
            self.measured_tension_n = sorted(self._recent_tension_readings)[2]

    async def update(self, dt):
        global running

        now = asyncio.get_event_loop().time()
        if self._last_update_time is None:
            real_dt = dt
        else:
            real_dt = now - self._last_update_time
        self._last_update_time = now

        if self.measured_tension_n > MAX_SAFE_TENSION_N:
            print(f">>> TENSION OVERLOAD ({self.measured_tension_n:.1f}N) - emergency stop!")
            await self.controller.set_stop()
            running = False
            return
```

```
113
114         position_error_turns = self.target_position_turns - self.position
115         position_error_mm = position_error_turns * CIRCUMFERENCE_MM
116
117         self.position_weight = min(1.0, abs(position_error_mm) / BLEND_DISTANCE_MM)
118         self.tension_weight = 1.0 - self.position_weight
119
120         if abs(position_error_turns) < POSITION_DEADBAND_TURNS:
121             position_correction = 0.0
122             self.position_integral = 0.0
123         else:
124             self.position_integral += position_error_turns * real_dt
125             self.position_integral = max(-POSITION_INTEGRAL_LIMIT, min(POSITION_INTEGRAL_LIMIT,
                      self.position_integral))
126             position_correction = (KP_POSITION * position_error_turns) + (KI_POSITION *
                      self.position_integral)
127
128         tension_error = self.target_tension_n - self.measured_tension_n
129
130         if abs(tension_error) < TENSION_DEADBAND_N:
131             tension_correction = 0.0
132             self.correction_start_time = None
133             self.tension_integral = 0.0
134             self._prev_tension_error = 0.0
135         else:
136             self.tension_integral += tension_error * real_dt
137             self.tension_integral = max(-TENSION_INTEGRAL_LIMIT, min(TENSION_INTEGRAL_LIMIT,
                      self.tension_integral))
138
139             tension_derivative = (tension_error - self._prev_tension_error) / real_dt if real_dt > 0
                      else 0.0
140             self._prev_tension_error = tension_error
141
142             tension_correction = (
143                 (KP_TENSION * tension_error)
144                 + (KI_TENSION * self.tension_integral)
145                 + (KD_TENSION * tension_derivative)
146             )
147
148             tension_correction = max(-TENSION_MAX_VELOCITY, min(TENSION_MAX_VELOCITY,
                      tension_correction))
149
150             now2 = asyncio.get_event_loop().time()
151             if self.correction_start_time is None:
152                 self.correction_start_time = now2
153             elif now2 - self.correction_start_time > CORRECTION_TIMEOUT_S:
154                 print(">>> CORRECTION TIMEOUT - stopping for safety!")
155                 await self.controller.set_stop()
156                 running = False
157                 return
158
159         if self.position_weight > 0.15:
160             self.tension_integral = 0.0
161
162         position_still_far = abs(position_error_mm) > (POSITION_DEADBAND_MM * 3)
163         corrections_oppose = (position_correction * tension_correction) < 0
164         tension_critical = self.measured_tension_n < MIN_SAFE_TENSION_N
165
166         if position_still_far and corrections_oppose and not tension_critical:
```

```
            suppression = min(1.0, max(0.0, (self.position_weight - 0.15) / 0.20))
            tension_correction *= (1.0 - suppression)

        raw_velocity = (self.position_weight * position_correction) + (self.tension_weight *
             tension_correction)
        self.velocity_cmd = max(-SAFE_VELOCITY, min(SAFE_VELOCITY, raw_velocity))

        if self.measured_tension_n < MIN_SAFE_TENSION_N and self.velocity_cmd < 0:
            self.velocity_cmd = 0.0

        if self.position >= POSITION_MAX_TURNS and self.velocity_cmd > 0:
            self.velocity_cmd = 0.0
        elif self.position <= POSITION_MIN_TURNS and self.velocity_cmd < 0:
            self.velocity_cmd = 0.0

        state = await self.controller.set_position(
            position=math.nan,
            velocity=self.velocity_cmd,
            accel_limit=ACCEL_LIMIT,
            velocity_limit=SAFE_VELOCITY,
            maximum_torque=MAX_TORQUE_LIMIT,
            query=True
        )

        self.actual_velocity = state.values[moteus.Register.VELOCITY]
        self.actual_torque_nm = state.values[moteus.Register.TORQUE]
        self.position = state.values[moteus.Register.POSITION]
        self.mode = state.values[moteus.Register.MODE]
        self.fault = state.values[moteus.Register.FAULT]

        if abs(self.actual_torque_nm) > STALL_TORQUE_THRESHOLD and abs(self.actual_velocity) <
             STALL_VELOCITY_THRESHOLD:
            self.stall_counter += 1
        else:
            self.stall_counter = 0

        if self.stall_counter > STALL_COUNT_LIMIT:
            print(">>> STALL DETECTED - stopping motor for safety!")
            await self.controller.set_stop()
            running = False

    def status_line(self):
        travel_mm = self.position * CIRCUMFERENCE_MM
        target_mm = self.target_position_turns * CIRCUMFERENCE_MM
        error_mm = target_mm - travel_mm
        return (

        )

async def read_arduino(ser, ctrl):
    global running
    print(">>> read_arduino started")
    try:
        while running:
            if ser.in_waiting > 0:
                data = ser.read(ser.in_waiting).decode('utf-8', errors='ignore')
                lines = data.strip().split('\r\n')
                if lines:
                    last_line = lines[-1].strip()
```

```
                    try:
                        grams = float(last_line)
                        ctrl.update_measured_tension(grams)
                    except ValueError:
                        pass
            await asyncio.sleep(0.01)
    except Exception:
        traceback.print_exc()

async def control_loop(ctrl):

    global running
    dt = 0.02
    counter = 0
    start_time = asyncio.get_event_loop().time()
    consecutive_errors = 0
    while running:
        try:
            await ctrl.update(dt)
            consecutive_errors = 0
        except Exception:

            traceback.print_exc()
            consecutive_errors += 1
            if consecutive_errors > 10:
                print(">>> Too many consecutive control loop errors - stopping for safety!")
                try:
                    await ctrl.controller.set_stop()
                except Exception:
                    pass
                running = False
                break

        counter += 1

        elapsed = asyncio.get_event_loop().time() - start_time
        travel_mm = ctrl.position * CIRCUMFERENCE_MM
        target_mm = ctrl.target_position_turns * CIRCUMFERENCE_MM

        log_times.append(elapsed)
        log_positions.append(travel_mm)
        log_target_positions.append(target_mm)
        log_tensions.append(ctrl.measured_tension_n)
        log_target_tensions.append(ctrl.target_tension_n)

        if counter % 5 == 0:
            print(ctrl.status_line())
        await asyncio.sleep(dt)

def save_graph_image():

    try:
        import matplotlib.pyplot as plt

        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

        ax1.plot(log_times, log_positions, label="Actual position", color="tab:blue")
        ax1.plot(log_times, log_target_positions, label="Target position", color="tab:blue",
             linestyle="--", alpha=0.5)
```

```
        ax1.set_ylabel("Position (mm)")
        ax1.legend()
        ax1.grid(True)
        ax1.set_title("Position over time")

        ax2.plot(log_times, log_tensions, label="Actual tension", color="tab:red")
        ax2.plot(log_times, log_target_tensions, label="Target tension", color="tab:red",
             linestyle="--", alpha=0.5)
        ax2.set_xlabel("Time (s)")
        ax2.set_ylabel("Tension (N)")
        ax2.legend()
        ax2.grid(True)
        ax2.set_title("Tension over time")

        plt.tight_layout()
        plt.savefig("convergence_diagram.png", dpi=150)
        print(">>> Graph saved as convergence_diagram.png")
    except ImportError:
        print(">>> matplotlib not installed: pip install matplotlib --break-system-packages")
    except Exception:
        traceback.print_exc()

async def get_user_input(ctrl):
    global running
    while running:
        val = await asyncio.get_event_loop().run_in_executor(None, input, "> ")
        parts = val.split()
        if not parts:
            continue
        if parts[0] == 'exit':
            running = False
            await ctrl.controller.set_stop()
            break
        elif len(parts) == 2 and parts[0] == 'pos':
            try:
                target = float(parts[1])
                ctrl.target_position_turns = ctrl.mm_to_turns(target)
                print(f">>> Target position: {target}mm")
            except ValueError:
                print(">>> Invalid number")
        elif len(parts) == 2 and parts[0] == 'ten':
            try:
                new_target = float(parts[1])
                if new_target > MAX_SAFE_TENSION_N:
                    print(f">>> Refused: exceeds safety limit of {MAX_SAFE_TENSION_N}N")
                else:
                    ctrl.target_tension_n = new_target
                    print(f">>> Target tension: {ctrl.target_tension_n}N")
            except ValueError:
                print(">>> Invalid number")

async def main():
    global running
    ser = serial.Serial(ARDUINO_PORT, ARDUINO_BAUD, timeout=0.01)
    await asyncio.sleep(2)

    controller = moteus.Controller()
    await controller.set_stop()
    await asyncio.sleep(0.1)
```

```
340
341     input(">>> Place the drum at ZERO and press Enter...")
342     await controller.set_output_exact(position=0.0)
343     await asyncio.sleep(0.1)
344     print(">>> Zero position set\n")
345
346     ctrl = WeightedBlendControl(controller)
347
348     asyncio.create_task(read_arduino(ser, ctrl))
349     asyncio.create_task(control_loop(ctrl))
350     await get_user_input(ctrl)
351
352     await controller.set_stop()
353     ser.close()
354
355     save_graph_image()
356
357 if __name__ == "__main__":
358     asyncio.run(main())
```

Listing 6.2: Hybrid controller

Appendix B — Component Datasheets

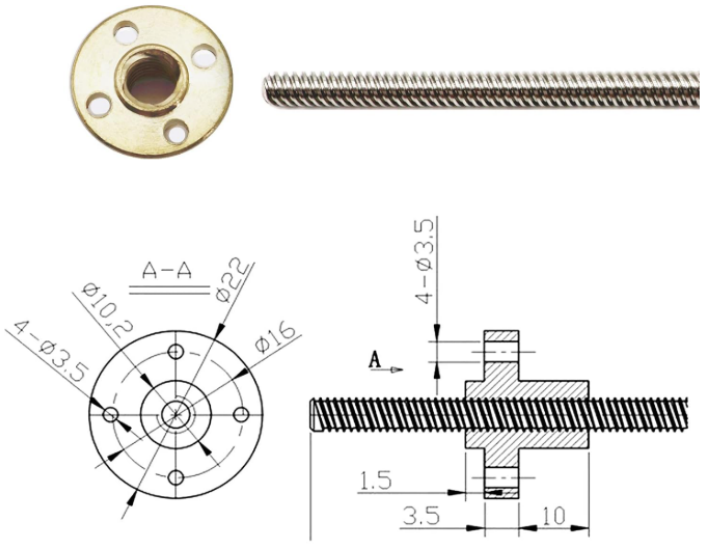


Figure B.1: Dimensions of the Tr8×2 lead screw and its brass nut [5]

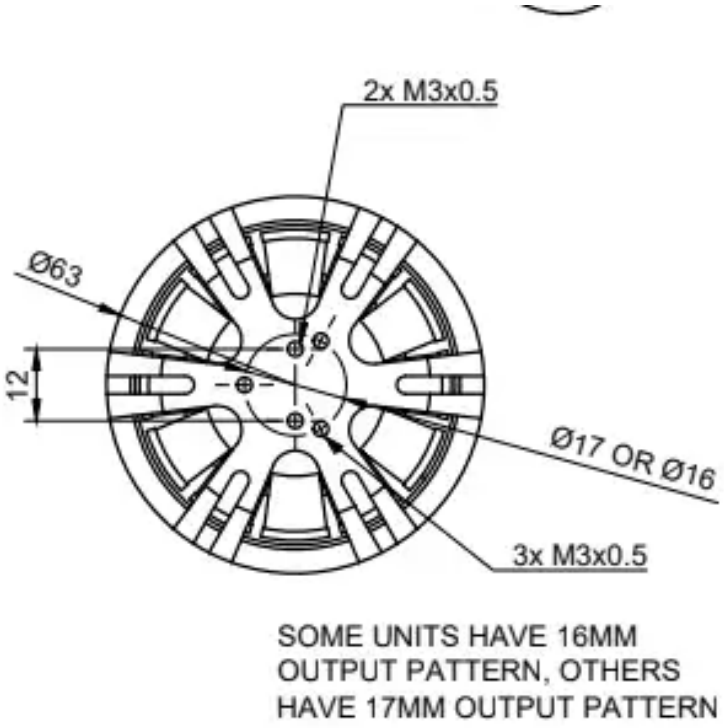


Figure B.2: Mechanical drawing of the mj5208 motor [1]

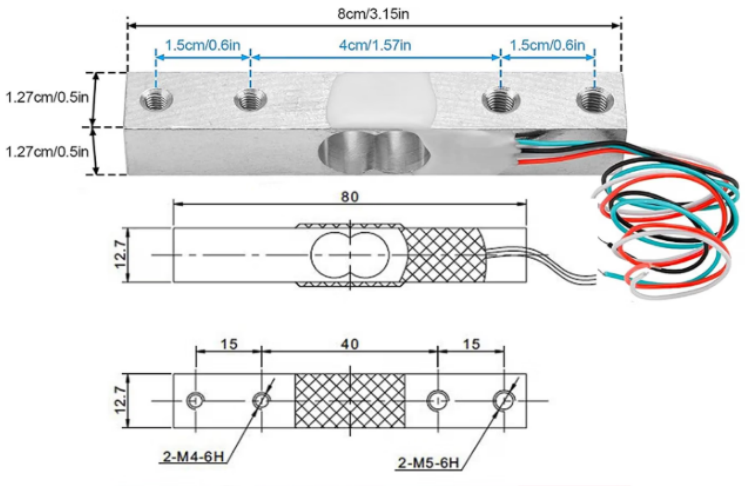


Figure B.3: Dimensions and characteristics of the load cell [4]

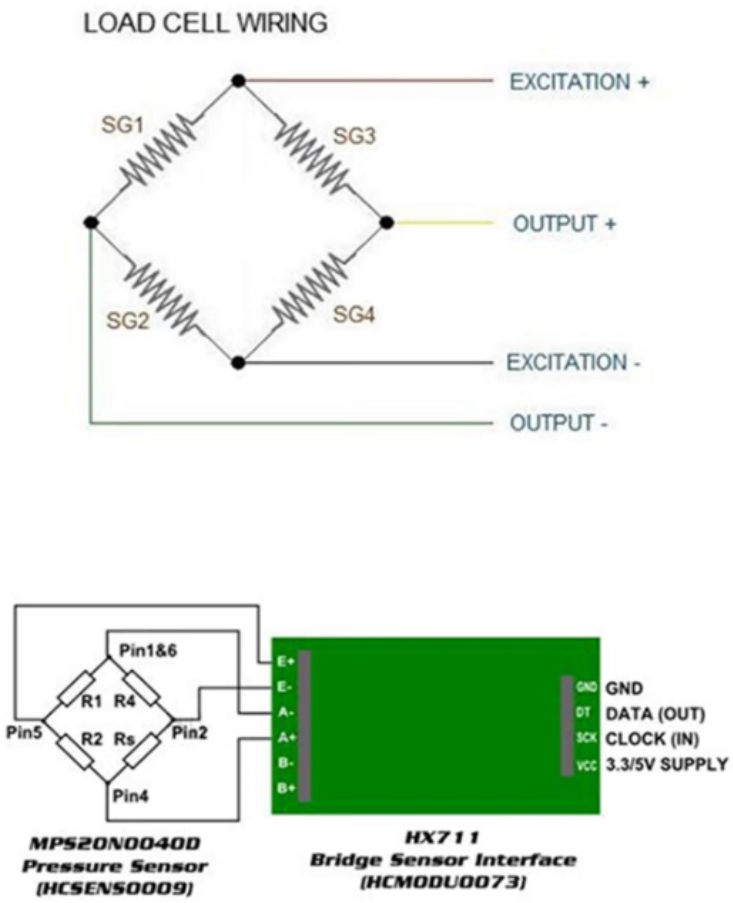


Figure B.4: Wiring of the load cell [4]